Introduction to Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI **4**
1.1 Rational decisions . . . 4
1.2 Agents and environments . . . 4
1.3 Methods and applications . . . 5
1.4 From an application to a computational problem . . . 6

2 Uninformed Search **7**
2.1 Planning and search problems . . . 7
2.2 State graphs and search trees . . . 7
2.3 Depth-first and breadth-first search . . . 8
2.4 Uniform-cost search . . . 9

3 A* Search and Heuristics **11**
3.1 Informed search . . . 11
3.2 Constructing heuristics . . . 11
3.3 Graph search and consistency . . . 12
3.4 Why closing states needs more care . . . 12

4 Constraint Satisfaction Problems I **14**
4.1 Representation and backtracking . . . 14
4.2 Filtering and arc consistency . . . 15
4.3 Ordering . . . 16

5 Constraint Satisfaction Problems II **17**
5.1 Consistency and structure . . . 17
5.2 Local search . . . 18
5.3 What local search keeps and what it loses . . . 18

6 Game Trees: Minimax **20**
6.1 Games and minimax values . . . 20
6.2 Alpha-beta pruning . . . 21
6.3 Depth limits and evaluation . . . 22

7 Game Trees: Expectimax and Utilities **23**
7.1 Chance nodes and opponent models . . . 23
7.2 Preferences over lotteries . . . 24
7.3 Money and risk . . . 25

8 Markov Decision Processes I **26**
8.1 States, rewards, and policies . . . 26
8.2 Bellman optimality and time-limited values . . . 26
8.3 Reading a Bellman backup . . . 27

9 Markov Decision Processes II **29**
9.1 Policy evaluation and extraction . . . 29
9.2 Policy iteration . . . 29
9.3 From planning to learning . . . 30

10 Reinforcement Learning I **31**
10.1 Learning a model or learning values . . . 31
10.2 Temporal differences . . . 31
10.3 Q-learning . . . 32

11 Reinforcement Learning II **34**
11.1 Exploration and regret . . . 34
11.2 Approximate Q-learning . . . 34
11.3 Policy search . . . 35

12 Probability **37**
12.1 Distributions, events, and conditioning . . . 37
12.2 Product, chain, and Bayes rules . . . 38
12.3 Inference by enumeration . . . 38

13 Bayes Nets: Representation **40**
13.1 Conditional independence . . . 40
13.2 Graph and local probabilities . . . 40
13.3 Representation and causality . . . 41

14 Bayes Nets: Independence **43**
14.1 Three local configurations . . . 43
14.2 D-separation . . . 44
14.3 Modeling consequences . . . 45

15 Bayes Nets: Inference **46**
15.1 Factors and elimination . . . 46
15.2 Evidence and normalization . . . 47
15.3 Ordering and complexity . . . 47

16 Bayes Nets: Sampling **49**
16.1 Prior and rejection sampling . . . 49
16.2 Likelihood weighting . . . 49
16.3 Gibbs sampling . . . 50

17 Decision Networks and Value of Information **52**
17.1 Decisions and expected utility . . . 52
17.2 Value of information . . . 53
17.3 Partial observability . . . 54

18 Hidden Markov Models **55**
18.1 Markov chains and stationary distributions . . . 55
18.2 Hidden states and observations . . . 55
18.3 Filtering and the forward algorithm . . . 56
18.4 A complete weather-filtering calculation . . . 57

19 Particle Filtering and Applications of HMMs **58**
19.1 Particles as a belief representation . . . 58
19.2 Dynamic Bayes nets . . . 59
19.3 Most likely state sequences . . . 59

20 Machine Learning: Naive Bayes **61**
20.1 Classification and the model . . . 61
20.2 Parameter estimation and smoothing . . . 61
20.3 Generalization and evaluation . . . 62

21 Machine Learning: Perceptrons and Logistic Regression **64**
21.1 Linear scores and perceptron updates . . . 64
21.2 Logistic probabilities . . . 65
21.3 Multiclass logistic regression . . . 66

22 Machine Learning: Optimization and Neural Networks **67**
22.1 Gradient optimization . . . 67
22.2 Learning representations . . . 68
22.3 Approximation and automatic differentiation . . . 68

23 Machine Learning: Neural Networks II and Decision Trees **70**
23.1 Induction, consistency, and simplicity . . . 70
23.2 Decision trees and expressiveness . . . 70
23.3 Information gain and recursive learning . . . 71
23.4 Overfitting and pruning . . . 72

24 Advanced Applications: Robotics, Vision, and Control **74**
24.1 Combining search and learning . . . 74
24.2 Helicopter estimation and control . . . 74
24.3 Locomotion and rewards . . . 75
24.4 Autonomous driving and manipulation . . . 76

25 Advanced Topics and Final Contest **77**
25.1 Cooperative game strategies . . . 77
25.2 Speech as sequential inference . . . 77
25.3 Connecting inference to action . . . 79

Lecture 1: Introduction to AI

1.1 Rational decisions

Artificial intelligence can be approached through four goals: making machines think like people, act like people, think rationally, or act rationally. This course focuses on the last: choosing actions that best achieve specified goals. A decision can be rational even if it does not resemble human thinking.

A rational decision maximizes expected utility, a numerical measure of the desirability of an outcome. If h denotes the information available when choosing an action, O is the set of possible outcomes, and $U(o)$ is the utility of outcome o, then for a finite outcome set the rule is

$$a^* \in \arg\max_a \sum_{o \in O} P(o \mid h, a)\, U(o).$$

For each action, weight each possible outcome's utility by its probability, then choose an action with the largest total. Rationality is relative to the goals and information available at decision time: an unlikely bad outcome can occur even when the chosen action had the highest expected utility. The criterion evaluates the decision, not whether its internal reasoning imitates a person.

The word *expected* matters. An agent usually chooses before it knows which outcome will occur. It should compare the whole distribution of outcomes for each action, not assume that the most likely outcome is certain. Utility also matters: a high probability of success is not enough if the cost of failure is unacceptable.

Example: As a small illustration, suppose a robot can use a safe route worth 6 utility units for sure, or a shortcut worth 10 with probability 0.7 and -5 otherwise. The shortcut's expected utility is $0.7(10) + 0.3(-5) = 5.5$, so the safe route is preferred. If the success probability rises to 0.9, the shortcut's value becomes 8.5 and the decision changes. The same actions can therefore lead to different rational choices when the information changes.

Rationality is evaluated using information available at the time. A decision can be sensible and still end badly. Conversely, a poor decision can get lucky. Judging only the eventual outcome confuses decision quality with chance. The agent's goal is another input: a route that is best for speed may be poor for energy use or safety.

1.2 Agents and environments

An agent perceives its environment and acts on it. Sensors deliver percepts, and actuators carry out actions. The environment then changes and supplies new percepts, so decisions form a continuing

interaction. The nature of the observations, available actions, and environment determines which techniques are appropriate.

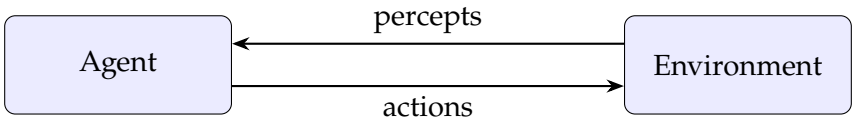


Figure 1: An agent interacts with its environment.

For Pac-Man, the environment is the game, percepts describe what the agent can observe, and actions move Pac-Man through the maze. The agent's objective specifies what counts as a good outcome; seeing the same game does not by itself determine what the agent should prefer.

A REFLEX AGENT chooses from its current percept, possibly using memory, without explicitly evaluating future consequences. A PLANNING AGENT uses a model to consider how candidate actions might change the world and selects a plan in light of its goal. A reflex rule can still be rational when it selects an optimal action for the setting; planning is one way to make decisions, not the definition of rationality.

The distinction between reflex and planning agents concerns *how an action is selected*, not whether the agent stores anything. A reflex agent can maintain an internal estimate of the world and still apply a rule to that estimate. A planning agent goes further: it predicts possible future states and compares them before choosing.

For example, a rule that turns away from a nearby wall can work well in a simple corridor. At a junction, choosing the branch that eventually reaches food requires comparing consequences beyond the next observation. Planning also has a cost. Looking farther ahead takes computation, and a detailed model can be expensive to use.

An explicit policy stores an action for each relevant situation. Once such a policy has been computed, using it can look like a reflex even if extensive planning was needed to obtain it. Thus the cost of constructing a policy and the cost of executing it are separate questions.

1.3 Methods and applications

AI systems combine computation and data. Search and planning compare possible actions; constraint satisfaction finds assignments that obey specified rules; adversarial and uncertain settings require reasoning about other agents or uncertain outcomes. Probabilistic models represent uncertainty, decision theory connects uncertainty to utility, and machine learning uses data to improve predictions or decisions.

The same principles appear in different applications:

- **Language:** speech recognition maps audio to text, speech synthesis maps text to audio, and

language systems support translation, question answering, and text classification.

- **Vision:** perception turns pixels into information, such as object identities, scene labels, or three-dimensional structure.
- **Robotics:** perception, planning, and control connect decisions to physical actions. The real world can differ from a simulator.
- **Games and logic:** game-playing systems reason over moves; logical systems use deduction, constraint satisfaction, and satisfiability to solve structured problems.

The field has used both knowledge-based and statistical approaches. Recent systems in the 2018 course draw on large datasets, computation, and neural networks; the appropriate representation and objective still depend on the task.

1.4 From an application to a computational problem

To build an agent, first make the decision problem explicit. What can it observe? Which actions can it take? What changes after an action? What outcomes does the objective reward? These choices determine what the agent must represent and what it can safely ignore.

In a maze, a route planner may need only locations and legal moves. If ghosts affect survival, their positions or a belief about their positions become relevant too. If the objective is to collect all food, remembering only the current location is insufficient because the remaining food depends on the past. Choosing a representation is therefore part of solving the problem, not merely a coding detail.

The major methods fit different gaps in this description. Search finds a good sequence when the model is known. Game search accounts for other decision makers. Probability represents unknown quantities. Learning estimates models or policies from examples and experience. A complete application often uses several of these methods together.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 1 and Note 1.

Lecture 2: Uninformed Search

2.1 Planning and search problems

A planning agent considers possible consequences before acting. It needs a model of how actions change the world and a goal test. A plan can be computed in advance or recomputed as new observations arrive. Search evaluates possible plans in the model, so its conclusions depend on the model's accuracy.

A search problem specifies a state space, a start state, a goal test, and a successor function. For a state, the successor function gives the available actions, resulting states, and step costs. A solution is a sequence of actions taking the start state to a goal; its cost is the sum of step costs. An optimal solution has minimum cost among all solutions.

A search state is an abstraction of the world: it retains the information needed to determine successors, costs, and goal status. For pathfinding in a maze, a position may suffice. To eat every dot, the state must also record which dots remain. If there are n possible positions and k dots, this representation has at most $n2^k$ states. Adding irrelevant details enlarges the search; omitting a detail that affects future actions or the goal makes the model inadequate.

The abstraction must contain enough information to predict the next search state. Two world situations may be merged only when their differences do not matter for the planned decisions. For pathfinding, facing direction can be omitted if turning is free and every direction is immediately available. If turning has a cost, it must be represented.

For the eat-all-dots problem, a state can be written (p, F), where p is the agent's position and F is the set of remaining food locations. Moving to p' gives the successor $(p', F \setminus \{p'\})$. Returning to the same position after eating a dot is a different state because F has changed. A search state should therefore describe the future-relevant situation, rather than merely where the agent is.

2.2 State graphs and search trees

In a state-space graph, each state is represented once, and an edge represents an action and its successor. A search tree unfolds the possible plans from the start: its root is the start and each child extends its parent's plan by one action. A tree node represents a path as well as its end state. Thus different nodes can end at the same state, and a cycle in a finite state graph can generate an infinite search tree. Both structures are usually constructed only as needed.

Tree search maintains a fringe of partial plans. It removes a node according to an exploration strategy, checks whether its state is a goal, and otherwise expands it by adding successor plans.

The strategy determines which plans are considered first. A goal is returned when selected from the fringe, which is especially important when path costs differ.

We compare strategies by COMPLETENESS, whether a solution is found when one exists, and OPTIMALITY, whether the returned solution has least cost. Let b be the maximum branching factor, m the maximum tree depth, and s the depth of the shallowest goal. These tree-search bounds count nodes, including repeated states; assume finite b.

A node normally stores its end state, a pointer to its parent, the action used to reach it, and its cumulative cost. Parent pointers reconstruct the answer after a goal is found: follow them to the root and reverse the actions. There is no need to copy an entire path into every node.

Algorithm 2.1: Tree search with a chosen fringe rule

```
Input: Start state s_0, successor function, and a fringe data structure
Insert a root node for s_0 into the fringe
while the fringe is not empty do
    Remove a node n according to the fringe rule
    if n ends at a goal then
        return the actions on n's parent chain
    for each legal successor (a, s', c) of n's state do
        Insert a child with state s', action a, and cost g(n) + c
return failure
```

The fringe represents alternatives still available. Expansion removes one alternative and replaces it with longer plans. A state can occur many times in this tree because different plans may reach it. This is why complexity in terms of tree depth can be large even when the underlying graph is small.

2.3 Depth-first and breadth-first search

DEPTH-FIRST SEARCH (DFS) expands a deepest fringe node first, using a last-in, first-out stack. It can pursue one branch indefinitely, so tree search is not complete for an infinite-depth tree. With finite depth it takes $O(b^m)$ time in the worst case. Its fringe holds at most the unexplored siblings along a root-to-leaf path, requiring $O(bm)$ space. DFS need not find a shallowest or least-cost solution.

BREADTH-FIRST SEARCH (BFS) expands a shallowest fringe node first, using a first-in, first-out queue. It explores by depth and is complete when branching is finite and a solution exists at finite depth. It finds a shallowest goal, which is optimal when all step costs are the same positive constant. Its time and space are exponential in solution depth; a conservative bound for the usual goal test on removal is $O(b^{s+1})$, because expansions at depth s before the goal can generate depth $s+1$ nodes.

ITERATIVE DEEPENING runs depth-limited DFS repeatedly with increasing limits. It finds a shallowest goal with the space advantage of DFS. Although it revisits upper levels, for $b > 1$ most nodes lie near the deepest level, so the repeated work changes the asymptotic time by only a constant factor.

Example: Consider a root S with children A, B, and let A have children C, D. If successors are

listed left to right, BFS selects $S, A, B, C, D, \ldots$. DFS follows one branch, selecting $S, A, C, \ldots$ before returning to the alternatives. A stack may need successors inserted in reverse order to realize that left-to-right traversal. The order among equal-priority nodes is a tie-breaking choice.

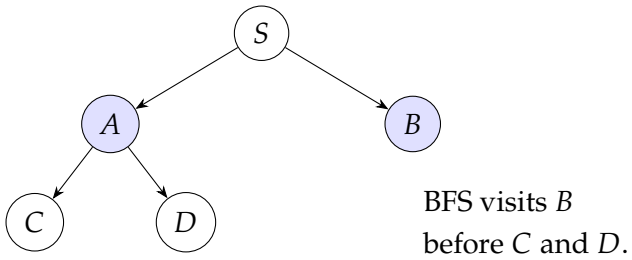


Figure 2: Depth and fringe order determine which partial plan is considered next.

BFS can be costly in memory because an entire wide layer remains available at once. DFS saves memory because it keeps only alternatives along its current branch, but that branch may never end. Iterative deepening trades repeated work near the root for a depth-first memory requirement. With a limit of L, a depth-limited search stops extending a branch at depth L; failure at that limit does not mean there is no deeper solution.

2.4 Uniform-cost search

When step costs differ, the fewest actions need not yield the cheapest plan. Uniform-cost search (UCS) uses a priority queue ordered by the cumulative path cost g. It removes a cheapest partial plan, tests for a goal, and expands it if necessary. It explores in increasing cost contours, without using information about the direction of the goal.

Suppose branching is finite, every step costs at least $\epsilon > 0$, and an optimal goal has finite cost C^*. UCS is complete: only finitely many tree nodes can have cost at most C^*, since their depth is at most C^*/ϵ. It is also optimal. If a more expensive goal were removed first, a frontier prefix of an optimal path would have cost at most C^* and would have been selected before that goal. This argument relies on nonnegative step costs and on checking the goal when it is removed, not merely when it is generated. Time and space are exponential in the effective depth C^*/ϵ, with implementation-dependent boundary factors from generating the next level.

For a small cost illustration, suppose $S \to G$ costs 10, while $S \to A \to G$ costs $1 + 1$. BFS can prefer the single-edge route, but UCS first removes A at cost 1, then the goal through A at cost 2. The goal at cost 10 may already be in the queue; its presence does not justify stopping. The cheapest *complete* plan is known only when no cheaper unfinished plan can lead to a better answer.

The positive lower bound on step costs is needed for tree-search completeness: infinitely many ever-cheaper extensions could otherwise keep a goal waiting. Optimality and completeness are distinct claims. Nonnegative costs support the optimality argument, while a positive lower bound plus finite branching ensures only finitely many relevant prefixes.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 2.

Lecture 3: A* Search and Heuristics

3.1 Informed search

A heuristic $h(s)$ estimates the remaining cost from state s to a nearest goal. Greedy search expands the node with smallest h, ignoring the cost already paid. It can be fast when the estimate points toward a goal, but can follow a misleading route and need not find a least-cost solution.

A* search instead orders the fringe by $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of the path represented by node n. With $h = 0$ it is uniform-cost search. A goal is tested when removed from the fringe: the first goal generated need not be the cheapest.

A heuristic is admissible if $0 \le h(s) \le h^*(s)$, where h^* is the true least remaining cost. In particular, h is zero at goals. For finite branching and step costs bounded below by a positive constant, A* tree search with an admissible heuristic is complete and optimal. To see optimality, suppose a suboptimal goal of cost $C > C^*$ is on the fringe. Some prefix n of an optimal path is also on the fringe until the optimal goal is removed. Admissibility gives $f(n) \le C^* < C = f(\text{goal})$, so the suboptimal goal cannot be removed first.

The score $g(n)$ looks backward: it is the cost already paid to reach this node. The score $h(n)$ looks forward: it estimates the cost still needed. Thus $f(n)$ estimates the total cost of a solution passing through this particular plan. Two nodes ending in the same state can have the same heuristic but different g values.

As an illustration, a frontier node with $g = 4, h = 3$ has $f = 7$, whereas one with $g = 1, h = 8$ has $f = 9$. A* selects the former even though reaching the latter has so far been cheaper. Greedy search would ignore that difference in past cost; UCS would ignore the estimated distance still to go.

The proof of optimality uses a *lower bound*, not a promise that the heuristic is accurate. For a prefix n of an optimal path,

$$g(n) + h(n) \le g(n) + h^*(n) = C^*.$$

For a suboptimal goal B, $f(B) = g(B) > C^*$. While that goal waits in the fringe, some unfinished prefix of the optimal path has a smaller score. This is the reason an admissible estimate cannot hide every optimal route behind a more expensive goal.

3.2 Constructing heuristics

A relaxed problem removes restrictions on actions; its optimal cost is a lower bound on the original cost and therefore an admissible heuristic. In the sliding-tile puzzle, the number of misplaced tiles is a lower bound on moves. The sum of the tiles' Manhattan distances to their goal positions

is another: each move changes one tile's distance by at most one. It corresponds to relaxing the restrictions imposed by the other tiles and the blank.

If $h_1(s) \geq h_2(s)$ everywhere, h_1 DOMINATES h_2. A more informative admissible estimate generally avoids expanding nodes with $f > C^*$, though ties and computation cost matter. The maximum of admissible heuristics remains admissible. The exact remaining cost is ideal as a bound, but computing it can be as hard as the original problem.

For the sliding-tile puzzle, the blank is not counted as a tile. A misplaced tile must move at least once, which explains the first lower bound. For Manhattan distance, a tile two columns and one row away contributes three. Since a legal move shifts just one tile by one grid step, the total distance can decrease by at most one per move. That is why the sum is a lower bound.

Taking $\max(h_1, h_2)$ keeps the strongest available lower bound at each state. Adding two admissible heuristics is not generally safe: they may both count the same necessary work. The heuristic itself also costs time, so fewer expansions need not imply less total computation.

3.3 Graph search and consistency

Tree search may revisit a state through many paths. Graph search keeps a closed set of expanded states and skips later occurrences. With costs, closing a state reached by a poor path can discard a better path discovered later; admissibility alone does not prevent this in A*.

A heuristic is CONSISTENT if for every edge $s \to s'$ of cost c,

$$h(s) \leq c(s, s') + h(s'), \qquad h(g) = 0 \text{ at goals.}$$

This triangle inequality implies f cannot decrease along a path. It also implies admissibility by summing along a path to a goal. With a consistent heuristic, the first expansion of a state is along a cheapest path to it: a frontier prefix of any cheaper path would have no greater f than that cheaper arrival, and hence would be expanded first. Consequently A* graph search that closes states on expansion is optimal. An alternative for an admissible but inconsistent heuristic is to reopen a state when a cheaper path is found.

3.4 Why closing states needs more care

A closed set should record states already *expanded*, not simply every state ever generated. For cost-based search it is also useful to record the best known g for each state and ignore stale queue entries.

The consistency inequality has a direct operational meaning:

$$f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n).$$

A child's priority cannot be better than its parent's. An optimal arrival at a state therefore cannot be overtaken by a more expensive arrival that causes the optimal one to be discarded.

An admissible but inconsistent estimate can misorder intermediate arrivals. For example, let $S \to A$ and $S \to B$ each cost 1, $A \to C$ cost 1, $B \to C$ cost 2, and $C \to G$ cost 3. The values $h(A) = 4$, $h(B) = 1$, $h(C) = 1$ are admissible. A* can expand B, then close C with $g(C) = 3$, before expanding A. If the later arrival at C with cost 2 is discarded, the answer costs 6 instead of 5. Reopening C when its cost improves fixes this failure. Consistency would forbid $h(A) = 4 > 1 + h(C) = 2$.

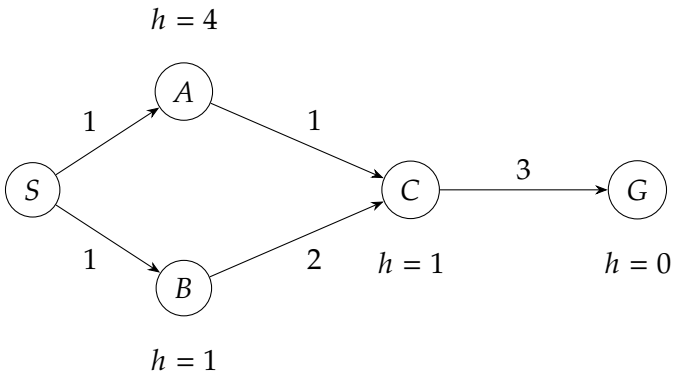


Figure 3: An inconsistent heuristic can let the costlier route to C arrive first.

Source. Pieter Abbeel and Dan Klein, Introduction to Artificial Intelligence, Fall 2018, Lecture 3.

Lecture 4: Constraint Satisfaction Problems I

4.1 Representation and backtracking

A constraint satisfaction problem (CSP) consists of variables $X_1, \ldots, X_n$, domains D_i of possible values, and constraints on allowed combinations of values. A solution assigns every variable a value and satisfies all constraints. Unlike path planning, the assignment itself is the goal. A constraint may be unary, binary, or involve more variables; it can be stated as a rule or a list of allowed tuples.

For map coloring, variables are regions, domains are colors, and adjacent regions must differ. In a binary CSP the constraint graph has a vertex for each variable and an edge for each pair constrained together. Disconnected components are independent subproblems. With n variables and at most d values each, there can be d^n complete assignments.

Backtracking search performs depth-first search over partial assignments. At each step it picks one unassigned variable and tries its values, rejecting an assignment as soon as it violates a constraint. Assignments commute, so it is unnecessary to branch over all possible orders in which the same variables could be assigned. If no value works, undo the most recent choice and try another. Filtering and ordering can reduce the search without changing its set of solutions.

Example: For n-queens, one useful variable is X_i, the row of the queen in column i, with domain $\{1, \ldots, n\}$. Different columns are built into this representation. The constraints for $i \neq j$ are $X_i \neq X_j$ and $|X_i - X_j| \neq |i - j|$, excluding shared rows and diagonals. This gives n variables instead of representing every square independently.

A backtracking call starts with a consistent partial assignment. It chooses one variable, tentatively assigns a value, checks constraints, and recurses. If recursion fails, it must undo both that assignment and any domain deletions caused by it. Failing to restore domains incorrectly rules out solutions in sibling branches.

Algorithm 4.1: Backtracking for a CSP

```
Input: Consistent partial assignment A and current domains D
if every variable is assigned then
    return A
Choose an unassigned variable X
for each candidate value v for X do
    if A ∪ {X = v} satisfies its applicable constraints then
        Save the current domains
        Assign X = v and propagate constraints
        if no domain is empty then
            Recursively search from the new assignment
            if a solution is found then
                return that solution
        Undo the assignment and restore saved domains
return failure
```

4.2 Filtering and arc consistency

Forward checking removes from each unassigned neighbor's domain values inconsistent with a new assignment. An empty domain proves failure, but forward checking does not detect every conflict between unassigned variables.

A directed arc $X \to Y$ is arc consistent if every value in D_X has a supporting value in D_Y satisfying the constraint between them. To revise an arc, delete unsupported values from the *tail* domain. To propagate constraints, repeatedly revise arcs; whenever D_X shrinks, recheck arcs from neighbors into X. An empty domain proves that the current branch has no solution. Domains must be restored on backtracking. Arc consistency can be enforced before search and after each assignment.

Arc consistency is not sufficient for global satisfiability. For example, three pairwise unequal variables forming a triangle, each with domain $\{\text{red}, \text{blue}\}$, are arc consistent but cannot all be assigned. Propagation therefore complements rather than replaces search. A basic implementation takes $O(n^2 d^3)$ time in the worst case; more efficient support tracking can improve this bound.

Consider $X \neq Y$ with $D_X = \{r, g\}$ and $D_Y = \{r\}$. The value r in D_X has no supporting value in D_Y, so revising $X \to Y$ removes it. The surviving g does have support, namely $Y = r$. Revising an arc means checking *each candidate in the tail*, not deleting matching values arbitrarily from either endpoint.

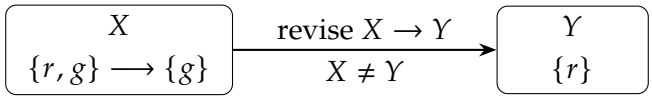


Figure 4: The head domain supplies support; unsupported values are deleted from the tail.

A queue makes propagation systematic. Start with arcs needing revision. When revising $X \to Y$

shrinks D_X, insert incoming arcs $Z \to X$ from other neighbors Z: a value in D_Z may have relied on a value just removed from D_X. Continue until the queue is empty or a domain becomes empty. No removed value could occur in a solution extending the current branch, because it had no compatible support.

Each arc can be reconsidered after domain changes. With at most d values, a simple revision checks up to d^2 pairs and an arc may be requeued $O(d)$ times. Over $O(n^2)$ possible arcs, this yields the basic $O(n^2 d^3)$ bound. Sparse graphs have fewer arcs, but the exponential worst case of the surrounding search remains.

4.3 Ordering

The MINIMUM REMAINING VALUES rule chooses an unassigned variable with the smallest current domain, exposing failure early. The LEAST CONSTRAINING VALUE rule tries the value that eliminates the fewest choices from other variables. The first rule chooses where to branch; the second chooses which branch to try first. Evaluating these choices and propagating constraints has a cost, traded against the search it saves.

MRV and least-constraining value serve different purposes. Choosing a nearly impossible variable first avoids spending time on choices that will later be undone. Once a variable is chosen, trying a flexible value first makes a successful completion more likely. The former asks where failure is most likely; the latter asks which value leaves the most room for success. Filtering changes the current domains, so these choices should use the updated domains rather than only their original sizes.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 4.

Lecture 5: Constraint Satisfaction Problems II

5.1 Consistency and structure

A CSP is k-consistent if any consistent assignment to $k-1$ variables can be extended to any kth variable. It is strongly k-consistent if it is also j-consistent for every $1 \leq j < k$. Strong n-consistency permits a solution to be built one variable at a time without backtracking, but enforcing higher-order consistency can be expensive.

The constraint graph can make a problem easier. Separate connected components can be solved independently. If n variables split into components of at most c variables, each with domain size at most d, exhaustive search costs roughly $O((n/c)d^c)$ for equal-sized components rather than $O(d^n)$.

A binary CSP whose constraint graph is a tree can be solved in $O(nd^2)$. Choose a root and order parents before children. Process children in reverse order, making each parent-to-child arc consistent by deleting unsupported parent values. Then assign the root and proceed forward, choosing for each child a value compatible with its parent. The backward pass guarantees support for each surviving parent value; once a child has been processed, later revisions do not remove its support. Thus the forward pass needs no backtracking. An empty domain signals failure.

For a nearly tree-structured graph, choose a cutset whose removal leaves a tree or forest. Try each assignment to the cutset, reject inconsistent ones, restrict the remaining domains, and solve the resulting forest. A cutset of size c gives a bound of order $d^c(n-c)d^2$, apart from checking the cutset constraints. Tree decompositions use overlapping groups of variables arranged in a tree, with agreement on shared variables.

Arc consistency checks support one neighbor at a time. It does not require all those supports to coexist in a single assignment. In the two-color triangle, $X = r$ has support $Y = g$ and support $Z = g$, but those two supports violate $Y \neq Z$. Higher-order consistency addresses this gap by requiring compatible extensions of assignments to several variables together.

The tree algorithm avoids that conflict because different child subtrees do not reconnect. Suppose a rooted chain has domains $D_X = \{r, g\}$, $D_Y = \{r, g\}$, $D_Z = \{r\}$ with inequality constraints. Processing $Y \to Z$ leaves $D_Y = \{g\}$; processing $X \to Y$ then leaves $D_X = \{r\}$. Assigning forward gives (r, g, r). There is no later edge between X and Z that could invalidate this choice.

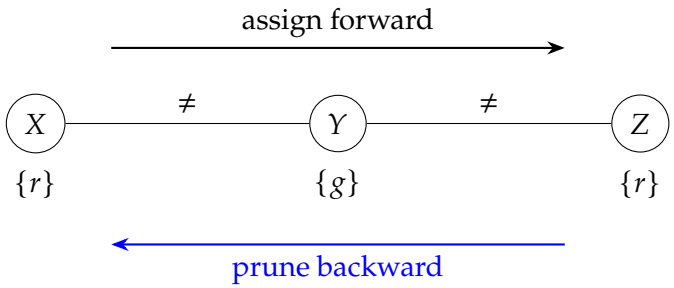


Figure 5: Pruning from leaves to root makes a forward assignment safe.

For cutset conditioning, assigning a variable means fixing its value and reducing its neighbors' domains before solving the remaining graph. Try all consistent cutset assignments; the method is still complete. A small cutset is valuable because the exponential dependence is on its size rather than on every variable.

Tree decomposition uses a different representation: groups of original variables become larger variables. Overlapping groups must agree on their shared assignments, and groups containing a given original variable must stay connected in the decomposition tree. Otherwise distant groups could give that variable conflicting values without detecting it. The cost moves into the domains of these larger groups.

5.2 Local search

Local search keeps a single complete assignment and changes it rather than maintaining a fringe of partial assignments. For a CSP, MIN-CONFLICTS selects a variable involved in a violated constraint and assigns it a value that minimizes the number of conflicts. It can be effective on large problems, but can become stuck or cycle and is not generally complete.

More generally, HILL CLIMBING repeatedly moves to a better neighbor and stops when none is better. Local optima and plateaus can prevent it from reaching a global optimum. SIMULATED ANNEALING sometimes accepts a worse move: for objective decrease $\Delta < 0$, a common acceptance probability is $e^{\Delta/T}$. As temperature T falls, downhill moves become less likely. Slow cooling has theoretical convergence guarantees, but escape can still be slow. Genetic algorithms select a population by fitness and create candidates using crossover and mutation. Their usefulness depends on the representation.

5.3 What local search keeps and what it loses

Backtracking remembers alternatives; local search normally remembers only the current complete assignment. For queens, a move changes the row of a queen within its column. Min-conflicts selects a queen currently attacking another queen and tests which row would leave the fewest attacks.

Unlike backtracking, it may temporarily keep violated constraints while repairing the assignment.

A local optimum is a state with no improving neighbor even though a better state exists elsewhere. On a plateau, many neighbors have equal scores, so the score gives little direction. Random choices, restarts, or occasional worse moves can help escape these situations, but ordinary hill climbing does not prove that no solution exists when it stops.

In simulated annealing, let Δ be the new score minus the current score for a maximization problem. Accept improving moves; for $\Delta < 0$, accept with probability $e^{\Delta/T}$. If a move loses 2 score units, its acceptance probability is e^{-1} at $T = 2$, but only e^{-4} at $T = 0.5$. The same loss becomes less acceptable as the search cools. A theoretical slow-cooling guarantee should not be confused with a fast practical algorithm.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 5.

Lecture 6: Game Trees: Minimax

6.1 Games and minimax values

A deterministic game specifies states, legal actions, a transition function, whose turn it is, a terminal test, and terminal utilities for each player. A solution is a policy that chooses an action in each state. In a two-player zero-sum game, one player's gain is the other's loss. Write all values from MAX's perspective; MIN minimizes that same value.

The minimax value is the best utility MAX can guarantee against an optimal adversary. For successors $\mathrm{Succ}(s)$,

$$V(s) = \begin{cases} U(s), & s \text{ terminal}, \\ \max_{s' \in \mathrm{Succ}(s)} V(s'), & \text{MAX's turn}, \\ \min_{s' \in \mathrm{Succ}(s)} V(s'), & \text{MIN's turn}. \end{cases}$$

Choose a successor attaining the maximum at MAX's root. The recursion follows from each player choosing its best continuation. A minimax strategy is safe against optimal opposition, but can be excessively cautious against an opponent whose errors could be exploited. For branching factor b and depth m, full minimax takes $O(b^m)$ time and $O(bm)$ space with depth-first traversal.

A terminal value is an outcome specified by the game. A nonterminal minimax value is computed from the values below it. At MAX nodes ask which move MAX would choose; at MIN nodes ask which move the opponent would choose. Do not average an adversary's moves unless the model assigns probabilities to those moves.

For a root with two MIN children, suppose the left child leads to utilities $8, 2$ and the right child to $5, 6$. Their values are $\min(8, 2) = 2$ and $\min(5, 6) = 5$. MAX chooses the right branch and can guarantee 5. Choosing the branch containing 8 would assume that MIN cooperates.

If the opponent has several consecutive turns, use one minimizing layer for each turn. The recursion depends on whose choice a node represents, rather than assuming that every second level must always be MAX.

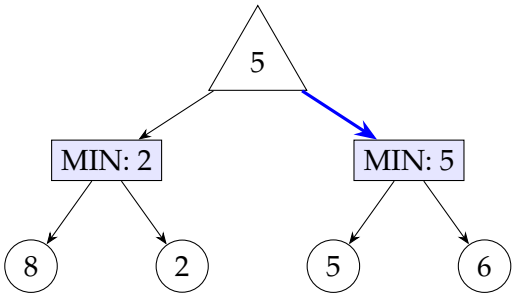


Figure 6: Back values up from leaves before choosing the root action.

6.2 Alpha-beta pruning

Alpha-beta pruning avoids branches that cannot affect the root's minimax decision. Carry α, the best alternative already available to MAX along the current path, and β, the best alternative available to MIN. Initialize them to $-\infty$ and $+\infty$.

At a MAX node, accumulate the largest child value, update α, and stop if the running value is at least β: an ancestor MIN already has an option no larger than β. At a MIN node, accumulate the smallest child value, update β, and stop if it is at most α. Unexamined children cannot change the ancestor's choice. A returned value after a cutoff can be only a bound on that internal node's true value; it should not be treated as an exact evaluation of every root child.

Pruning preserves the root value. Its effectiveness depends on ordering: examine promising moves first so useful bounds become available early. The worst case remains $O(b^m)$; ideal ordering reduces the leaf count to order $b^{m/2}$, permitting roughly twice the depth for the same budget.

To see a cutoff, suppose MAX has already found a root move worth 3. It begins exploring another MIN child and sees a leaf worth 2. That MIN child's final value is at most 2, whatever its other leaves contain. MAX can already obtain 3, so this entire branch is irrelevant to its decision. The bound is enough; the exact minimum is unnecessary.

The symmetric case occurs below a MIN ancestor. If MIN can already choose a branch worth 5 and another MAX branch reaches a child worth 7, MAX's final value is at least 7. MIN will avoid it, so its other children need not be searched. Always compare values in the same utility convention, from MAX's point of view.

The root action should be stored when its running best value improves. A cutoff in some other child need not produce that child's exact value, but it does establish that the child cannot improve the retained choice. Good move ordering finds strong alternatives early and creates more cutoffs.

6.3 Depth limits and evaluation

Realistic trees are too large to reach terminal states. At a depth limit, replace terminal utility by an EVALUATION FUNCTION, often $\text{Eval}(s) = \sum_i w_i f_i(s)$ for features such as material advantage or distance to food. This removes the optimal-play guarantee: search now optimizes the truncated model and its estimates.

Greater depth can reduce sensitivity to evaluation error, but expensive features leave less time to search. Iterative deepening provides a completed shallower result while deeper search proceeds. Evaluation also guides move ordering for alpha-beta. Replanning with a short horizon can postpone a good action forever when taking it now and taking it just beyond the horizon look equally valuable; the evaluation must distinguish useful progress from delay.

A PLY is one player's move. Cutting off after two plies means considering one's action and one reply, not necessarily two complete rounds. At the cutoff, an evaluation function tries to summarize what happens beyond the horizon. It should reward progress that the search will otherwise postpone and recognize danger that lies just outside the search depth.

Suppose eating a dot now and eating it one move later produce the same score at the cutoff. A replanning agent may repeatedly choose to delay, since every new search again makes delay look harmless. Adding a feature that values proximity to remaining food can distinguish those choices. This is an evaluation problem, not a failure of the minimax recursion itself.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 6.

Lecture 7: Game Trees: Expectimax and Utilities

7.1 Chance nodes and opponent models

Uncertain outcomes may come from random events, action failures, or an opponent whose behavior is modeled probabilistically. Expectimax uses MAX nodes for choices and chance nodes for outcomes:

$$V(s) = \max_a V(s,a), \qquad V(s,a) = \sum_{s'} P(s' \mid s,a)V(s').$$

At terminal nodes use utility; at a depth cutoff use an estimate of future expected utility. For example, outcomes $8, 24, -12$ with probabilities $1/2, 1/3, 1/6$ have expectation $4 + 8 - 2 = 10$.

The probabilities encode a model, even when the opponent does not literally randomize. A model of an opponent that usually chooses a minimax move can put most mass on that move and the remainder on alternatives. Computing such a model may itself require search. A wrong model can be costly: optimism about an adversarial opponent can be dangerous, while worst-case reasoning about a random opponent can miss worthwhile opportunities.

An unexplored chance outcome can have an arbitrarily large value, so ordinary alpha-beta cutoffs do not apply without bounds on utilities. Games with both opposition and chance use MAX, MIN, and chance layers as appropriate. With multiple independent utilities, a node can carry a utility vector and the player to move chooses the child maximizing its own component, with a specified rule for ties.

In a chance node, every outcome contributes to the value. Probabilities must be nonnegative and sum to one. They describe outcomes conditional on the current state and selected action, so using the same probabilities in every state can be a serious modeling error.

For a small illustration, action A gives 4 for sure. Action B gives 10 with probability $1/2$ and 0 otherwise. Expectimax prefers B, whose value is 5; a worst-case MIN model prefers A, since B's worst outcome is 0. Neither recursion determines which model is appropriate. That depends on whether the uncertainty actually behaves like the stated distribution or like an opponent selecting the worst outcome.

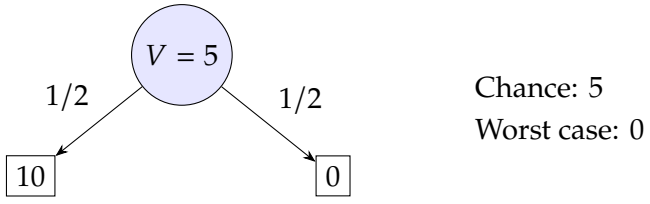


Figure 7: Chance takes a probability-weighted average; MIN takes the worst child.

At a depth limit, the evaluation should estimate expected future utility under the same opponent or environment model. A function tuned for worst-case danger can produce a cautious expectimax agent even though its recursion averages outcomes.

Ordinary pruning fails because an unfinished weighted average can rise or fall as remaining outcomes are revealed. If utilities are known to lie in $[L, H]$ and unexamined probability mass is p, their remaining contribution lies in $[pL, pH]$. Bounds can then be used, but the first bad child alone does not justify an alpha-beta-style cutoff.

7.2 Preferences over lotteries

A LOTTERY is a probability distribution over outcomes. Rational preferences satisfy constraints including completeness, transitivity, continuity, substitutability, monotonicity in the probability of a preferred prize, and reduction of compound lotteries to their outcome probabilities. Under the expected-utility representation, an outcome utility U can be chosen so that a lottery L is valued by

$$U(L) = \sum_o P_L(o)U(o).$$

The maximum expected utility principle chooses the action with greatest expected utility given the available information. A rational policy need not explicitly calculate this expression; its decisions can implement the same preferences by another mechanism.

Minimax decisions are unchanged by any strictly increasing transformation of terminal utilities because only order matters. Expectimax depends on the relative spacings: only positive affine transformations $U' = aU + b$, $a > 0$, preserve all lottery preferences. Normalize best and worst outcomes to 1 and 0; an outcome indifferent to a lottery paying the best outcome with probability p then has utility p.

The preference assumptions rule out contradictory choices. Completeness lets the agent compare two lotteries. Transitivity prevents a cycle in which it prefers A to B, B to C, and C to A. Continuity allows an intermediate prize to be matched by an appropriate lottery between better and worse prizes. Substitution allows indifferent lotteries to replace one another inside larger lotteries.

Why does utility scale matter? In the example above, taking square roots changes the sure outcome's value from 4 to 2, while the risky action's expected value becomes $\frac{1}{2}\sqrt{10} \approx 1.58$. The preferred action reverses. This increasing transformation keeps outcome rankings but changes preferences between lotteries. For a positive affine change,

$$\mathbb{E}[aU + b] = a\mathbb{E}[U] + b,$$

so the action ordering is preserved. This calculation is the reason the allowed transformations are narrower for expected utility.

7.3 Money and risk

Expected money and expected utility are different objectives. A concave utility of wealth expresses risk aversion: the utility of a sure mean can exceed the mean utility of a lottery. A CERTAINTY EQUIVALENT is the sure amount judged equivalent to a lottery; its difference from expected money measures the amount the agent would pay to remove the risk. Human preferences need not satisfy the axioms. In the Allais example, the common choices \$3000 for sure over an 80% chance of \$4000, and a 20% chance of \$4000 over a 25% chance of \$3000, imply contradictory inequalities for any single expected-utility function with $U(0) = 0$.

For the Allais choices, preferring the certain \$3000 gives $U(3000) > 0.8U(4000)$. Preferring the 20% chance of \$4000 gives $0.2U(4000) > 0.25U(3000)$, or $0.8U(4000) > U(3000)$. These cannot both hold. The inconsistency concerns the combination of preferences, not whether either individual gamble is inherently sensible.

Risk aversion does not mean always avoiding uncertain actions. It means evaluating them with a utility curve whose gains from additional money become smaller. Expected utility can still favor a risky action when its potential benefits are large enough.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 7.

Lecture 8: Markov Decision Processes I

8.1 States, rewards, and policies

A Markov decision process (MDP) specifies states S, actions $A(s)$, transitions $T(s, a, s') = P(s' \mid s, a)$, rewards $R(s, a, s')$, a start state, and possibly terminal states. The Markov assumption says that the current state and action determine the distribution of the next state, without additional dependence on history.

In a noisy grid world, a requested move can go in the intended direction with probability 0.8 and each perpendicular direction with probability 0.1; a blocked move leaves the state unchanged. A living reward and terminal rewards express preferences between speed, safety, and remaining in the grid. The solution is a policy $\pi : S \to A$ rather than a fixed sequence: the action must respond to the state actually reached.

For discount $0 \le \gamma < 1$, the return from rewards $r_0, r_1, \ldots$ is $G = \sum_{t \ge 0} \gamma^t r_t$. With bounded rewards $|r_t| \le R_{\max}$, $|G| \le R_{\max}/(1 - \gamma)$. Discounting favors earlier rewards and makes infinite-horizon values finite. Finite horizons instead end after a fixed number of steps, and the best action can depend on time remaining. Undiscounted episodic problems need conditions ensuring termination and finite returns.

A policy must cover outcomes the agent did not intend to produce. If a move north can slip west, the agent needs an action for the westward destination too. Writing down only a path assumes away that uncertainty.

Rewards specify the objective; they are not the same as values. A reward is received on one transition. A value includes the expected rewards of an entire continuation. Giving a state a large positive reward can make it attractive, but a nearby state can also have high value because it leads there reliably.

For rewards $[1, 2, 3]$ and $\gamma = 1/2$, return is $1 + (1/2)2 + (1/4)3 = 2.75$. The exponent counts how many steps in the future a reward arrives. A reward already included in $R(s, a, s')$ must not be counted again as a terminal continuation value. Otherwise the terminal payoff is accidentally doubled.

8.2 Bellman optimality and time-limited values

Let $V^*(s)$ be the optimal expected return starting at s, and $Q^*(s, a)$ the expected return after committing to action a and then acting optimally. Decomposing the return into the next reward

and the discounted remainder gives

$$Q^*(s,a) = \sum_{s'} T(s,a,s')\big[R(s,a,s') + \gamma V^*(s')\big], \qquad V^*(s) = \max_a Q^*(s,a).$$

Terminal continuation values are zero when their rewards have already been paid on entry. These equations reuse values of repeated states instead of unfolding a separate infinite expectimax tree for every path.

Value iteration begins with $V_0(s) = 0$ and computes

$$V_{k+1}(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')].$$

Here V_k is the best value with k steps remaining. Use the previous vector throughout a synchronous iteration. A dense update costs $O(|S|^2|A|)$; sparse transition models require summing only over possible successors.

For a finite-depth problem, enough iterations recover the full value. For bounded rewards and $\gamma < 1$, the tail beyond depth k has magnitude at most $\gamma^k R_{\max}/(1-\gamma)$. Thus the truncation error vanishes and V_k converges to V^*. The maximizing actions can stabilize much earlier than the numerical values.

8.3 Reading a Bellman backup

The backup has a fixed order of operations. For each possible first action, compute an average over its possible next states. In each term add the immediate reward to the discounted continuation. Only after computing each action's expected return do we choose the largest one.

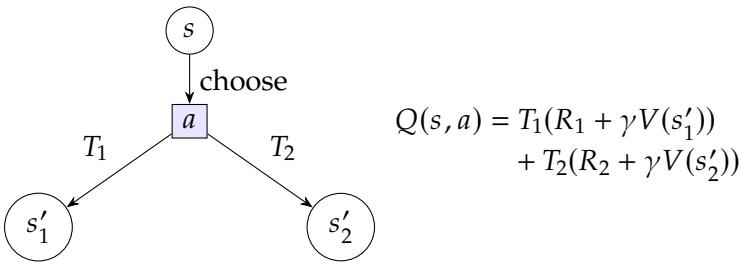


Figure 8: Average outcomes within an action, then compare actions.

Example: In an illustrative backup, action a reaches a state of old value 6 with probability 0.8 and one of value 1 with probability 0.2. If both immediate rewards are -1 and $\gamma = 0.9$, its value is $0.8(-1 + 0.9 \cdot 6) + 0.2(-1 + 0.9 \cdot 1) = 3.5$. Compare 3.5 with the corresponding averages for the other actions.

Algorithm 8.1: Synchronous value iteration

Set $V(s) = 0$ for every state
repeat
  for each nonterminal state s **do**
    $V_{\text{new}}(s) \leftarrow \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V(s')]$
  Set terminal entries of V_{new} to zero
  Replace V by V_{new}
until the chosen convergence condition is met

Each pass adds one more step of lookahead to a finite-horizon calculation. Using a separate new vector makes that interpretation exact. Updating entries in place is a different update schedule, even if it can also converge.

A one-state example makes the limiting process visible. If every step pays 2, returns to the same state, and $\gamma = 1/2$, then $V_0 = 0$, $V_1 = 2$, $V_2 = 3$, $V_3 = 3.5$, and the limit solves $V = 2 + V/2$, giving $V = 4$. The values improve gradually because more of the future reward stream has been included.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 8.

Lecture 9: Markov Decision Processes II

9.1 Policy evaluation and extraction

For a fixed deterministic policy π, the policy value is $V^\pi(s) = \mathbb{E}_\pi[\sum_{t\geq 0} \gamma^t r_t \mid s_0 = s]$. Fixing the action removes the maximum from the Bellman equation:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Compute it by iterating this update, at $O(|S|^2)$ per dense pass, or by solving the linear system $(I - \gamma T_\pi)V^\pi = r_\pi$. For a finite MDP with $\gamma < 1$, the system has a unique solution.

Policy extraction from a value estimate requires one-step lookahead:

$$\pi_V(s) \in \arg\max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V(s')].$$

Choosing the successor with largest value alone ignores rewards and transition probabilities. From optimal action values, extraction is simply $\pi^*(s) \in \arg\max_a Q^*(s, a)$.

The transition matrix T_π has one row for each state and one column for each possible next state. Its entry is $T(s, \pi(s), s')$. The vector r_π contains the expected immediate rewards $r_\pi(s) = \sum_{s'} T(s, \pi(s), s')R(s, \pi(s), s')$. Thus $V^\pi = r_\pi + \gamma T_\pi V^\pi$ is just the collection of the one-state evaluation equations. Solving them together accounts for loops.

One-step policy extraction is not the same as choosing the neighbor with largest V. An action may reach a good state only rarely, charge a large immediate cost, or occasionally fall into a bad state. The full weighted backup includes all three effects.

9.2 Policy iteration

Policy iteration alternates two operations: evaluate the current policy to obtain V^π, then replace it by the greedy policy π_{V^π}. The improvement step cannot lower value: it chooses a first action at least as good as the old policy's action, followed by continuation values under the old policy. Repeated application of the improved policy preserves that inequality. If no action changes, the values satisfy Bellman optimality and the policy is optimal. Retain the old action on ties to avoid cycling between equally good policies.

Value iteration updates values and implicitly reselects every action each pass. Policy iteration spends several cheaper passes evaluating one fixed policy before reconsidering actions. Both solve the same known MDP; the computational advantage depends on the problem and evaluation

method.

Example: Consider an illustrative one-state MDP with discount 1/2. Action a returns to the state and pays 1. Action b returns too and pays 3 with probability 1/2, otherwise 0. Under the policy always taking a, $V^\pi = 1 + V^\pi/2 = 2$. Improvement compares $Q^\pi(s, a) = 1 + (1/2)2 = 2$ with $Q^\pi(s, b) = 1.5 + (1/2)2 = 2.5$. It switches to b. Evaluating this new policy gives $V = 1.5 + V/2 = 3$.

In that comparison the continuation after the first action is still the *old* policy. Policy improvement asks whether changing the first action is useful when the remaining values are those already evaluated. Once the new policy is adopted everywhere, another evaluation computes the effects of all its future choices.

To justify improvement, let $\mathcal{B}_{\pi'}$ mean taking one backup under the new greedy policy. Then $\mathcal{B}_{\pi'} V^\pi \geq V^\pi$. Applying the same monotone backup repeatedly keeps the inequality, and its limit is $V^{\pi'}$. Therefore $V^{\pi'} \geq V^\pi$. If the greedy improvement changes nothing, the current evaluation is also a solution of the optimality equations.

Policy iteration can save time because many value updates do not require comparing every action again. Exact evaluation can nevertheless be expensive, so there is no universal claim that policy iteration is always faster. The model's size, sparsity, and how accurately policies are evaluated matter.

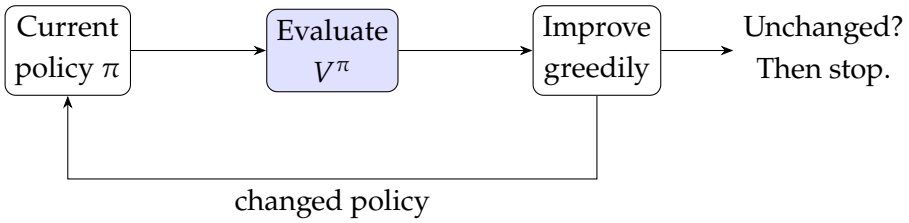


Figure 9: Policy iteration alternates evaluating one policy and choosing a better one.

9.3 From planning to learning

Offline planning computes values and policies from known transitions and rewards without interacting with the environment. When those quantities are unknown, an agent must act and observe. Repeated outcomes estimate their probabilities, but learning incurs mistakes. Exploration gathers information about uncertain actions; exploitation uses current knowledge for reward. Learning a good final policy and collecting good rewards during learning are different objectives. The double-bandit example highlights the missing information in learning. With known reward probabilities, expected values can be computed before playing. If those probabilities change or are unknown, calculation alone cannot reveal them. The agent needs observations from actions. An unlucky short sequence does not prove an action is poor, so repeated trials are needed to separate chance from a real difference in expected reward.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 9.

Lecture 10: Reinforcement Learning I

10.1 Learning a model or learning values

Reinforcement learning retains the MDP objective but learns from observed transitions (s, a, r, s') instead of being given T and R. Model-based learning counts successor outcomes to estimate $\widehat{T}(s, a, s') = N(s, a, s')/N(s, a)$ and estimates rewards from observations. It then solves the estimated MDP. Unvisited state-action pairs need further experience before their model can be trusted.

In passive learning, the policy is fixed and the task is evaluation. Direct evaluation averages observed discounted returns following visits to each state. It is model free and simple, but learns states separately rather than exploiting known estimates of their successors; it also waits for a return to become available.

In a learned model, counts answer a local question: among the times action a was taken in state s, how often did each successor occur? If four observations of (s, a) lead to u, u, u, v, the transition estimates are $\widehat{T}(s, a, u) = 3/4$ and $\widehat{T}(s, a, v) = 1/4$. A reward estimate is an average if the same transition can have different rewards. Planning on this model still inherits errors from limited data.

Direct return averaging learns a different object. For an episode with rewards $-1, -1, 10$ and $\gamma = 1$, the return from the first state is 8, from the second is 9, and from the third is 10. Each observed return is a noisy sample of a policy value. The method does not automatically force two states with the same successor to share what was learned there.

10.2 Temporal differences

Temporal-difference learning updates on each transition under the fixed policy:

$$\delta = r + \gamma V(s') - V(s), \qquad V(s) \leftarrow V(s) + \alpha\delta.$$

The target is one observed reward plus an estimated continuation. Sampling outcomes automatically weights them by their transition probabilities in expectation. This bootstrapping shares information through the connections between states without estimating a transition table.

The update is an interpolation $(1 - \alpha)V(s) + \alpha[r + \gamma V(s')]$. A constant learning rate emphasizes recent samples; suitably decreasing rates allow averages to settle. Terminal continuation values are zero. TD evaluation alone does not supply a model-free improvement rule from state values: deciding among actions would still require their transition and reward models.

The TD error is a mismatch between two estimates of the same return. $V(s)$ is the current prediction before the transition. $r + \gamma V(s')$ is a new target using the reward just seen and the current prediction

at the successor. If the target is larger, increase $V(s)$; if it is smaller, decrease it.

For the transition $B \to C$ with reward -2, let $V(B) = V(C) = 0$, $\gamma = 1$, and $\alpha = 1/2$. The target is -2, so $V(B)$ becomes -1. Next observe $C \to D$ with reward -2 and $V(D) = 8$. The target is 6, so $V(C)$ becomes 3. A later visit to $B \to C$ can now use that improved estimate of C. Learning propagates backward through repeated experience.

A constant learning rate is not an ordinary average of all past targets. Expanding the recursion gives weights $\alpha, \alpha(1-\alpha), \alpha(1-\alpha)^2, \ldots$ on increasingly old targets. This can be useful while the targets themselves are changing, but it leaves continuing noise. Decreasing rates allow estimates to settle.

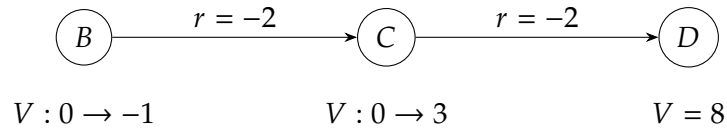


Figure 10: Each observed transition updates the prediction at its starting state.

10.3 Q-learning

Active learning chooses actions as well as evaluating them. The model-based Bellman update for action values is

$$Q_{k+1}(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')].$$

Q-learning replaces this expectation by sampled updates:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)].$$

Action selection then uses $\arg\max_a Q(s,a)$ without a transition model. Q-learning is off-policy: the greedy continuation in its target can differ from the exploratory behavior that collected the data. In the finite, discounted tabular setting with bounded rewards, sufficient repeated visits to every state-action pair and per-pair rates satisfying $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$ permit convergence to Q^*. Merely acting greedily from the start need not provide that coverage.

Q-learning stores one prediction for each possible first action. On a sample transition, only the entry $Q(s,a)$ is updated. The maximum at the successor means the target assumes the best future action according to current estimates; it does not require the behavior policy to take that action next.

For example, if $Q(s, a) = 4$, the observed reward is 1, the largest successor Q-value is 6, $\gamma = 0.5$, and $\alpha = 0.2$, then the target is 4 and there is no update. If the reward is instead 3, the target is 6 and the entry becomes $4 + 0.2(6 - 4) = 4.4$. This is one sample, not proof that the true action value is 6.

For a terminal successor, use a continuation of zero rather than maximizing over an empty action set. Continued exploration and suitable rates are part of the convergence assumptions. Off-policy learning does not mean that an agent can learn the value of actions it never tries.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 10.

Lecture 11: Reinforcement Learning II

11.1 Exploration and regret

An ϵ-greedy behavior policy selects a random action with probability ϵ and a currently greedy action otherwise. Persistent randomness provides exploration but keeps making avoidable mistakes after learning. Decreasing ϵ can reduce that cost, provided exploration still visits the relevant state-action pairs often enough.

An exploration function instead combines value and visit count into an optimistic score $f(u, n)$. For example, give a high score R^+ while a pair has fewer than N_e visits, then use its estimated value. Replacing $\max_{a'} Q(s', a')$ by $\max_{a'} f(Q(s', a'), N(s', a'))$ in a target can propagate the incentive to reach unexplored states backward. The bonus is an exploration device, not a change to the actual reward objective.

Regret measures the expected reward lost relative to optimal behavior over the learning period. Two methods can converge to the same optimal policy and still have very different regret because they make different mistakes on the way.

Exploration must sometimes pay a short-term cost to reach a useful unknown area. Randomly choosing among the immediate actions is easy, but it may be inefficient when learning requires several coordinated exploratory steps. An optimistic score gives the agent a reason to travel toward an uncertain state, not just to randomize after arriving there.

The number of visits matters separately for each state-action pair. An action observed once with a bad result is less well understood than one observed many times with consistently bad results. A bonus expresses that difference; it should gradually stop dominating evidence.

Regret counts the price paid while learning. In a simple bandit illustration, suppose the best action's expected reward is 1.5 and another action's is 1. Taking the second action on 20 trials costs $20(1.5 - 1) = 10$ expected reward units relative to always taking the better action. Individual realized rewards can differ by chance. Some of that loss may be worthwhile to discover which action is better, but unnecessary exploration adds cost after the distinction is already clear.

11.2 Approximate Q-learning

A table cannot generalize to unvisited states and can be too large to store. Represent a state-action pair by features $f_i(s, a)$, such as distances to food or ghosts, and use a linear approximation

$$Q_w(s, a) = \sum_i w_i f_i(s, a).$$

Experience now changes a shared set of weights. Define the TD error

$$\delta = r + \gamma \max_{a'} Q_w(s', a') - Q_w(s, a), \qquad w_i \leftarrow w_i + \alpha \delta f_i(s, a).$$

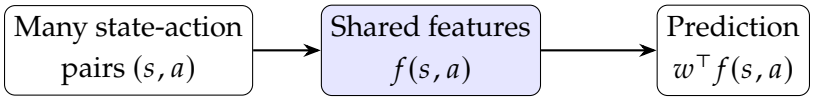
This is a gradient step on squared prediction error when the sampled target is held fixed: the derivative of the prediction with respect to w_i is $f_i(s, a)$. A negative TD error lowers the prediction for the current pair and changes estimates for other pairs sharing its features. With one indicator feature per pair, the rule recovers tabular Q-learning.

Generalization saves data and memory, but states with the same features may have different values. Capacity limits can reduce overfitting, while an inadequate feature representation can prevent a good policy. The tabular convergence guarantee does not automatically carry over to approximation.

Example: Suppose two features of a pair are $f_1 = 1$ and $f_2 = 2$, with weights $w_1 = 0.5, w_2 = 1$. Its prediction is $Q_w = 2.5$. If the sampled target is 1.5, then $\delta = -1$. With $\alpha = 0.1$, the weights become $w_1 = 0.4$ and $w_2 = 0.8$. The new prediction on that pair is 2.0. The larger second feature caused a larger change to its weight.

The same change affects any other pair with those features. A pair with features $(1, 0)$ changes from 0.5 to 0.4 even if it has never been visited. This is the benefit of generalization, and also its danger: shared features can make experiences interfere when the situations should have different values.

The squared-error interpretation treats the target as fixed during the update. For target y, define $L(w) = \frac{1}{2}(y - w^\top f)^2$. Then $\partial L / \partial w_i = -(y - w^\top f) f_i = -\delta f_i$. Gradient descent gives exactly $w_i \leftarrow w_i + \alpha \delta f_i$. This does not make the whole Q-learning process ordinary supervised regression, because later targets also depend on the changing weights.



One weight update changes many predictions.

Figure 11: Features tie different experiences to the same learned parameters.

11.3 Policy search

Accurate numerical value estimates are a means to choosing good actions; their ordering can matter more than their absolute errors. Policy search directly adjusts policy parameters to improve expected return. A simple approach perturbs feature weights and compares returns from sampled episodes. Random outcomes require repeated trials, and testing each parameter separately becomes expensive in high dimension. Robotic locomotion and manipulation illustrate why generalization and learning effective policies are needed when enumerating all states is infeasible.

For policy search, a parameter change is judged by the behavior it produces. Two Q-functions may have very different numerical errors and still select the same actions. Conversely, a small error that reverses the top two actions can matter greatly. This explains why improving value prediction and improving return are related but not identical tasks.

To compare two candidate policies, run enough episodes to estimate their returns rather than deciding from one lucky outcome. Direct search can avoid demanding accurate values everywhere, but it gives a noisier and often more expensive signal about whether a weight change was helpful.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 11.

Lecture 12: Probability

12.1 Distributions, events, and conditioning

A random variable represents an uncertain quantity and has a domain of possible values. A discrete probability distribution assigns nonnegative mass summing to one. A joint distribution assigns a probability to every complete assignment of a collection of variables. With n variables of domain size d, its table has d^n entries.

An event is a set of assignments; its probability is the sum of their masses. To obtain a marginal, sum out variables not of interest: $P(x) = \sum_y P(x, y)$. For $P(e) > 0$, conditioning restricts attention to assignments satisfying the evidence and renormalizes:

$$P(x \mid e) = \frac{P(x, e)}{P(e)}, \qquad P(e) = \sum_x P(x, e).$$

A distribution such as $P(X \mid e)$ is a table over X; an expression with a specified lowercase value, $P(x \mid e)$, is a number. A family $P(X \mid Y)$ contains a normalized distribution for every value of Y.

For the joint masses $P(\text{hot}, \text{sun}) = 0.4$, $P(\text{hot}, \text{rain}) = 0.1$, $P(\text{cold}, \text{sun}) = 0.2$, and $P(\text{cold}, \text{rain}) = 0.3$, marginalizing gives $P(\text{cold}) = 0.5$. Selecting the cold rows and dividing by 0.5 gives $P(\text{sun} \mid \text{cold}) = 0.4$.

It helps to distinguish three operations on a joint table. Selection removes rows inconsistent with evidence. Marginalization adds rows that differ only in variables being discarded. Normalization divides every retained value by their total. Selection alone does not make probabilities sum to one.

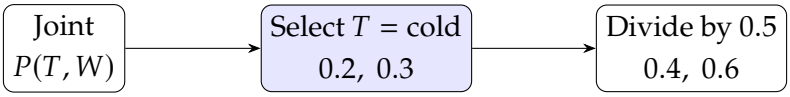


Figure 12: Conditioning first selects compatible outcomes, then rescales them.

The probability of an OR event needs care when its pieces overlap. In the weather table, $P(\text{hot or sun})$ includes the rows (hot, sun), $(\text{hot}, \text{rain})$, and $(\text{cold}, \text{sun})$, giving 0.7. Adding $P(\text{hot}) + P(\text{sun}) = 0.5 + 0.6$ counts the first row twice. Subtracting $P(\text{hot}, \text{sun}) = 0.4$ corrects that double count.

Conditioning is also directional. Here $P(\text{sun} \mid \text{cold}) = 0.4$, but $P(\text{cold} \mid \text{sun}) = 0.2/0.6 = 1/3$. They answer different questions and have different denominators.

12.2 Product, chain, and Bayes rules

The definition of conditioning gives the product rule $P(x, y) = P(x \mid y)P(y)$. Repeated application yields the chain rule, valid without independence assumptions:

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}).$$

Writing $P(x, e)$ in both orders gives Bayes' rule:

$$P(x \mid e) = \frac{P(e \mid x)P(x)}{\sum_{x'} P(e \mid x')P(x')}.$$

The prior $P(x)$ is reweighted by the likelihood $P(e \mid x)$ to obtain the posterior. A likely observation under a hypothesis does not by itself make the hypothesis likely: its prior and competing explanations also matter. For a noisy location sensor, multiply each candidate location's prior by the probability of the observed reading at that location, then normalize.

Example: Use a simple illustrative diagnostic model. A condition has prior probability 0.01. A positive test has probability 0.9 when the condition is present and 0.1 when it is absent. Then

$$P(+) = 0.9(0.01) + 0.1(0.99) = 0.108, \qquad P(\text{condition} \mid +) = \frac{0.009}{0.108} = \frac{1}{12}.$$

The test is much more likely to be positive for an affected case, but the many unaffected cases still contribute more positive results in total.

The denominator is a sum over every way the evidence can arise. Omitting the unaffected cases would turn the likelihood $P(+ \mid \text{condition})$ into an incorrect posterior. This is the common error of reversing a conditional probability without accounting for base rates.

The chain rule requires no assumption about which variables cause which. For three variables it reads $P(a, b, c) = P(a)P(b \mid a)P(c \mid a, b)$. Replacing the last term by $P(c \mid b)$ is an additional independence assumption, not another use of the chain rule.

12.3 Inference by enumeration

Partition variables into query Q, evidence $E = e$, and hidden variables H. Select the joint entries matching e, sum out H, and normalize:

$$P(Q \mid e) = \alpha \sum_{h} P(Q, h, e).$$

Here α is the reciprocal of the sum of the resulting entries, provided the evidence has positive probability. Enumeration is exact, but a full joint table needs exponential storage and generally exponential work. Structured models will express the joint using smaller local tables.

For example, let S be season, T temperature, and W weather. To query weather given $T = t$, sum out the hidden season:

$$P(W \mid T = t) = \alpha \sum_{s} P(s, t, W).$$

For each weather value, add the masses for both seasons. These sums describe weather together with the fixed temperature. Their total is $P(T = t)$. Dividing by it produces a distribution over weather alone.

If the evidence has probability zero under the model, that denominator is zero. The ordinary conditional formula supplies no answer; one must inspect whether the model or observation is inappropriate rather than divide by zero.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 12.

Lecture 13: Bayes Nets: Representation

13.1 Conditional independence

Two variables are independent when $P(x, y) = P(x)P(y)$ for all values. They are conditionally independent given Z, written $X \perp\!\!\!\perp Y \mid Z$, when, for all z of positive probability,

$$P(x, y \mid z) = P(x \mid z)P(y \mid z).$$

Equivalently, knowing Y adds no information about X once Z is known. Conditional independence need not imply unconditional independence. For example, two sensor readings can both depend on a ghost's location while being independent once the location is fixed.

To check conditional independence, the statement must hold for every relevant value of the conditioning variable, not merely for one convenient case. For the cavity example, suppose toothache and a probe catching are generated independently once cavity status is fixed. Knowing the toothache then adds nothing to the catch probability if cavity status is already known. Without that status, a toothache changes belief about the cavity, which in turn changes belief about the probe catching.

Independence saves parameters because a table over combinations can be replaced by smaller tables. It also commits the model to treating some information as redundant. The usefulness of the simplification depends on whether those commitments fit the application.

13.2 Graph and local probabilities

A Bayesian network is a directed acyclic graph with one random variable per node and a conditional probability table (CPT) $P(X_i \mid \mathrm{Pa}(X_i))$ per node. It defines

$$P(x_1, \ldots, x_n) = \prod_i P(x_i \mid \mathrm{pa}(x_i)).$$

Choose a topological order. The modeling assumption replaces all earlier variables in the chain rule by the node's parents: the node is independent of earlier nonparents given its parents. The product is normalized because variables can be summed out in reverse topological order, each CPT summing to one for fixed parent values.

In the alarm network, burglary B and earthquake E influence alarm A; the alarm influences calls J and M:

$$P(B, E, A, J, M) = P(B)P(E)P(A \mid B, E)P(J \mid A)P(M \mid A).$$

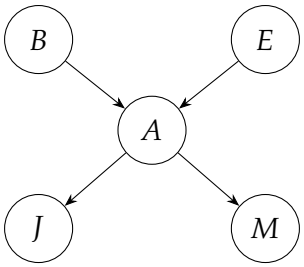


Figure 13: The alarm network's local dependencies. All variables are unobserved.

The absence of an edge does not say that two nodes can never be statistically related. Burglary can be related to a call through the alarm even though there is no direct edge from burglary to the call. Likewise, the graph does not say that an arrow guarantees a large numerical effect; the CPT determines how strongly parent values change a child's probabilities.

A network of independent coin flips has no edges and factors into individual priors. For n Boolean variables with at most k parents each, CPT storage is $O(n2^k)$ rather than $O(2^n)$ for a joint table. This saving depends on the graph's sparsity.

A CPT is a separate distribution for each combination of parent values. For Boolean A with two Boolean parents, there are four parent contexts. Within each context, $P(A = 1 \mid B, E)$ determines $P(A = 0 \mid B, E) = 1 - P(A = 1 \mid B, E)$. A root has no parent context and its CPT is simply a prior.

Example: In the alarm model, take $P(b) = 0.001$, $P(\neg e) = 0.998$, $P(a \mid b, \neg e) = 0.94$, $P(j \mid a) = 0.9$, and $P(m \mid a) = 0.7$. The probability of this *complete* assignment is

$$P(b, \neg e, a, j, m) = 0.001(0.998)(0.94)(0.9)(0.7) = 0.0005910156.$$

No summation is needed because every variable is assigned. Computing $P(b \mid j, m)$ is different: alarm and earthquake are not specified, so their possible values must be summed out before normalization.

For n Boolean variables with at most k parents, the number of independent CPT parameters is at most $n2^k$: one probability for each parent context. A full joint instead needs $2^n - 1$ independent parameters. This saving is large only when the local parent sets remain small.

13.3 Representation and causality

A topology restricts which distributions can be represented by requiring certain conditional independences. A fully connected acyclic graph can express any joint distribution through the chain rule. Causal ordering often makes the model easier to specify, but arrows in a general Bayes net assert a factorization, not necessarily causation. A two-variable joint can be factored in either direction, with appropriately changed priors and CPTs. Observational association alone does not establish which variable causes the other.

The acyclic requirement permits a topological order in which every parent comes before its child. Starting with the chain rule in that order explains the factorization. It also explains a way to generate samples: draw roots first, then draw each child given the parents already generated.

Reversing an edge can preserve a distribution only if the other tables are changed appropriately and the resulting graph still represents the required dependencies. Simply reversing an arrow while retaining the same numbers generally specifies a different model.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 13.

Lecture 14: Bayes Nets: Independence

14.1 Three local configurations

Independence is a property of a distribution. A graph guarantees some independences for every choice of its CPTs; a particular numerical model can have additional independences.

In a chain $X \to Y \to Z$, observing the middle variable blocks the connection: $X \perp\!\!\!\perp Z \mid Y$. Without that observation, changes in X can change beliefs about Y and hence Z. A fork $X \leftarrow Y \to Z$ behaves similarly: the effects can be associated through a shared cause, but become independent given that cause, in the isolated three-node model.

A collider $X \to Y \leftarrow Z$ behaves oppositely. With independent root causes, summing out Y leaves $P(X)P(Z)$, so $X \perp\!\!\!\perp Z$. Observing Y can couple them: evidence for one explanation can reduce the need for the other. Observing a descendant of Y can also activate this connection. This is EXPLAINING AWAY.



Figure 14: Shading marks observation. The arrow pattern determines its effect on a path.

Example: Consider a simple explaining-away model with independent rare causes X, Z, each having probability 0.1, and an effect Y that occurs exactly when at least one cause occurs. Before observing the effect, $P(X) = 0.1$. After $Y = 1$,

$$P(X = 1 \mid Y = 1) = \frac{0.1}{1 - 0.9^2} = \frac{10}{19}.$$

If $Z = 1$ is then learned, it already explains the effect, and $P(X = 1 \mid Y = 1, Z = 1) = 0.1$ again. Information about one cause changes belief about the other only after evidence couples them.

Evidence need not be at the collider itself. If Y causes a report D, observing D can tell us something about Y and thereby connect X and Z. A path test must therefore check observed descendants of colliders, not just the nodes lying directly on the path.

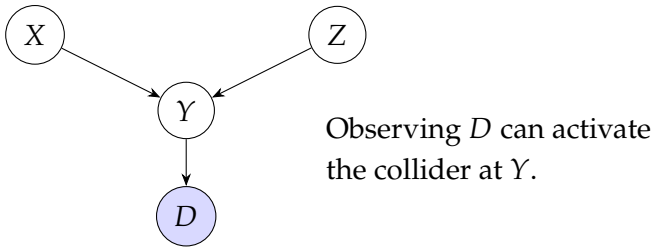


Figure 15: Evidence below a collider can open its path between the causes.

14.2 D-separation

To test whether X and Y are independent given evidence variables E, consider paths in the underlying undirected graph while retaining arrow directions to classify triples. A path is active exactly when:

- every noncollider on it is unobserved;
- every collider is observed or has an observed descendant.

One inactive triple blocks the entire path. If every path is blocked, X and Y are D-SEPARATED by E and $X \perp\!\!\!\perp Y \mid E$ in every distribution represented by the network.

An active path means independence is not guaranteed by the graph, not that dependence must occur for every CPT. Other paths must always be checked: blocking one route does not establish independence if another remains active. The rules depend on which nodes are observed, not on the numerical values of that evidence, for the graph's universal guarantees.

A practical way to apply the rule is to trace one path at a time and inspect each internal node using the two arrows on that path. A node is a collider only if both of those arrows point into it. A node with two incoming edges elsewhere in the graph may be a noncollider on the particular path being tested.

For the alarm graph, $B \rightarrow A \leftarrow E$ is blocked without evidence at A or its descendants, so burglary and earthquake are independent in the prior. Observing John calling can activate the collider because J is a descendant of A. In contrast, the route $B \rightarrow A \rightarrow J$ is a chain: conditioning on A blocks the connection between burglary and the call.

Do not interpret a blocked path as an instruction to erase its probabilities from the model. D-separation answers whether evidence about one variable can change another variable's distribution given the stated observations. Numerical probabilities still come from the complete model.

14.3 Modeling consequences

D-separation exposes assumptions made by selecting a topology. Adding edges can remove guaranteed independences and enlarge the represented family, but also enlarges CPTs and can make inference more expensive. The graph is a compact description of allowable distributions; numerical inference is a separate task requiring its CPTs.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 14.

Lecture 15: Bayes Nets: Inference

15.1 Factors and elimination

For query variables Q, hidden variables H, and evidence e,

$$P(Q \mid e) = \alpha \sum_{H} \prod_{i} P(X_i \mid \mathrm{Pa}(X_i))\big|_{E=e}.$$

A factor is a table over its unassigned variables; it need not be a normalized distribution. Selecting an observed value removes that dimension. Joining factors multiplies their entries over the union of their variables; summing out a variable adds entries that agree on all remaining variables.

Enumeration first constructs the joint and then marginalizes. Variable elimination interleaves these operations. To eliminate hidden variable H, collect every factor containing H, multiply them, sum out H, and return the resulting factor to the collection. Factors not containing H remain unchanged. After eliminating all hidden variables, multiply remaining factors and normalize over Q.

The justification is distributivity: factors independent of the summed variable can be pulled outside its sum. For the chain $R \to T \to L$,

$$P(L) = \sum_{t} P(L \mid t) \underbrace{\sum_{r} P(r)P(t \mid r)}_{f(t)}.$$

The inner result is stored once for each t, avoiding a full table over R, T, L. With $P(r) = 0.1$, $P(t \mid r) = 0.8$, and $P(t \mid \neg r) = 0.1$, $f(t) = 0.17$. If $P(l \mid t) = 0.3$ and $P(l \mid \neg t) = 0.1$, then $P(l) = 0.17(0.3) + 0.83(0.1) = 0.134$ and $P(\neg l) = 0.866$.

The current factor collection is the algorithm's memory. Eliminating H replaces only the factors containing H because other factors are constant with respect to its sum. If those factors are $f_1(H, A)$ and $f_2(H, B)$, their replacement is

$$g(A, B) = \sum_{h} f_1(h, A) f_2(h, B).$$

Although H disappears, its effect survives as a dependence between A and B. A newly created factor is generally not a CPT for an original node.

Algorithm 15.1: Variable elimination for $P(Q \mid e)$

```
Start with all CPTs, selecting the observed values e
for each hidden variable H in the chosen order do
    Collect factors F_H whose scope contains H
    Form g = Σ_H Π_{f∈F_H} f
    Remove F_H and insert g into the collection
Multiply the remaining factors to obtain u(Q)
return u(Q)/Σ_Q u(Q), if the denominator is positive
```

A factor's *scope* is its set of unassigned variables. Multiplying factors requires matching common-variable values before multiplying entries. It does not mean multiplying arbitrary rows position by position. Summing out a variable groups entries with equal assignments on the remaining scope.

15.2 Evidence and normalization

Instantiate evidence in every factor containing an observed variable before elimination. Preserve the query variables. The final unnormalized table is $P(Q, e)$, whose sum is $P(e)$. Dividing by this sum gives the posterior. Normalizing an intermediate factor without tracking its scale can lose an evidence likelihood, so retain factors as unnormalized tables until the desired final normalization.

Continue the traffic example with the evidence $L = l$. Selecting that evidence turns $P(L \mid T)$ into a factor over T with entries $0.3, 0.1$. First eliminating T gives

$$g(r) = 0.8(0.3) + 0.2(0.1) = 0.26, \qquad g(\neg r) = 0.1(0.3) + 0.9(0.1) = 0.12.$$

Multiplying by the rain prior produces masses 0.026 and 0.108. Their sum is 0.134, so

$$P(r \mid l) = \frac{0.026}{0.134} = \frac{13}{67}.$$

The likelihood of being late changes the prior rain probability from 0.1 to about 0.194. The result is not $P(l \mid r) = 0.26$, since the query conditions in the opposite direction.

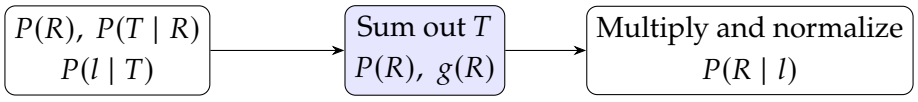


Figure 16: Eliminate the hidden variable while keeping the query variable.

15.3 Ordering and complexity

A factor over k variables with domains of size d has d^k entries. Thus time and space depend critically on the largest intermediate factor, which depends on elimination order. Eliminating a highly connected variable early can join many otherwise separate variables into a large table. The

answer is independent of order; the amount of computation is not.

There is no universally efficient ordering: general Bayes-net inference is NP-hard. One can encode a satisfiability problem so that a designated event has positive probability exactly when a satisfying assignment exists. Sparse structures such as polytrees admit efficient inference relative to their local table sizes. Cutset conditioning trades enumeration over selected variables for simpler remaining graphs, paralleling the structural methods for CSPs.

Elimination order changes which variables become coupled in intermediate factors. Eliminating the center of a star can combine all its leaves in one table; eliminating the leaves first can keep each intermediate table small. With k binary variables in a scope, there are 2^k entries, so one added variable can double a table's size.

A compact input network therefore does not guarantee cheap exact inference. Representation size depends on original CPTs; inference cost depends on the intermediate factors needed for the particular query and order. Evidence can shrink scopes by fixing variables and sometimes makes an otherwise difficult query easier.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 15.

Lecture 16: Bayes Nets: Sampling

16.1 Prior and rejection sampling

To sample a finite distribution, partition $[0, 1)$ into intervals whose lengths equal its probabilities, draw a uniform number, and select its interval. Prior sampling applies this in topological order, drawing each node from its CPT given its sampled parents. The probability of a full sample is the product of these conditionals, exactly the network's joint.

Estimate an event by its frequency among samples. To estimate $P(Q \mid e)$, rejection sampling discards samples inconsistent with the evidence and tallies query values in the rest. Accepted samples have the desired conditional distribution. If $P(e)$ is small, only about $NP(e)$ of N prior samples survive; impossible evidence yields no posterior estimate.

If $n(q, e)$ is the number of accepted samples with query value q, rejection sampling uses $n(q, e)/n(e)$. Dividing by the total number of original draws would estimate $P(q, e)$ instead of $P(q \mid e)$. This distinction is the same normalization step used in exact inference.

If evidence has probability 0.001, then 100,000 prior draws yield only about 100 accepted samples on average. The apparent size of the simulation is misleading: the conditional estimate is based on those accepted draws. One can stop a draw as soon as it contradicts evidence, saving the remaining sampling work, but this does not improve the acceptance probability.

Prior samples are drawn from the model, not from the unknown real world. They approximate the probabilities already encoded in the CPTs. More samples reduce simulation error; they do not correct a poorly specified model.

16.2 Likelihood weighting

Likelihood weighting fixes evidence variables to their observed values and samples all other variables in topological order. Initialize $w = 1$. At an evidence node $E_i = e_i$, multiply w by $P(e_i \mid \text{pa}(E_i))$ instead of drawing its value. Return the complete assignment and its weight, and normalize weighted query counts:

$$\widehat{P}(q \mid e) = \frac{\sum_{j=1}^{N} w_j \mathbf{1}\{Q_j = q\}}{\sum_{j=1}^{N} w_j}.$$

Writing z for sampled variables, the proposal contains the non-evidence CPTs, while the weight contains the evidence CPTs:

$$q(z) = \prod_{i \notin E} P(z_i \mid \mathrm{pa}_i), \qquad w(z) = \prod_{i \in E} P(e_i \mid \mathrm{pa}_i), \qquad q(z)w(z) = P(z, e).$$

This identity explains consistency of the weighted estimate. Evidence affects downstream draws immediately, but does not guide upstream draws; a few samples can therefore dominate the total weight.

For the cloudy-sprinkler-rain-wet-grass network, suppose sprinkler and wet grass are observed true. A likelihood-weighted draw might sample cloudy true and rain false. The two fixed observations contribute

$$w = P(S = 1 \mid C = 1)P(W = 1 \mid S = 1, R = 0) = 0.1(0.9) = 0.09.$$

Another draw can receive a different weight because it supplies different parents for those same observations. It is the weighted count, rather than the raw number of draws, that contributes to the query estimate.

Fixing evidence does not mean pretending it was certain under the original model. Its likelihood remains in the weight. If that correction were omitted, upstream variables would be drawn according to their prior regardless of how well they explain the evidence.

Very unequal weights waste effort: a hundred tiny-weight samples can contribute less than one large-weight sample. All-zero weights mean that the current sample collection cannot explain the evidence, so its normalized estimate is undefined. This can happen with finite samples even when the evidence is possible in the full model.

16.3 Gibbs sampling

Gibbs sampling keeps a complete assignment consistent with the evidence and repeatedly resamples one non-evidence variable given all the others. For node X,

$$P(x \mid \text{rest}) \;\propto\; P(x \mid \mathrm{pa}(X)) \prod_{Y \text{ child of } X} P(y \mid \mathrm{pa}(Y)).$$

All other CPTs cancel because they do not vary with x. Consequently the parents, children, and other parents of the children supply the needed local information. Both upstream and downstream observations can influence a draw.

Under conditions ensuring that this Markov chain explores the posterior's support and converges, long-run frequencies estimate the posterior. Successive samples are dependent, and slow mixing or deterministic constraints can make convergence difficult. More samples improve accuracy only insofar as they provide useful exploration of the distribution.

To see a Gibbs update, fix $C = 1, R = 0, W = 1$ and resample S. The two unnormalized possibilities are

$$u(S = 1) = P(S = 1 \mid C = 1)P(W = 1 \mid S = 1, R = 0) = 0.09,$$
$$u(S = 0) = P(S = 0 \mid C = 1)P(W = 1 \mid S = 0, R = 0) = 0.009.$$

Normalizing gives $P(S = 1 \mid C = 1, R = 0, W = 1) = 10/11$. The factors $P(C)$ and $P(R \mid C)$ cancel because changing S does not change their entries. Rain and wet grass both matter through the child's CPT, even though rain is not a parent of S.

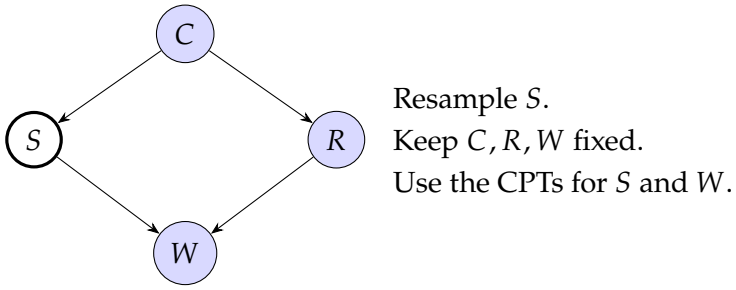


Figure 17: A Gibbs update uses only factors that change with the resampled variable.

After updating S, choose another unobserved variable and repeat. Unlike prior sampling, this need not follow a topological order. The current assignment is gradually changed while preserving the posterior as the stationary target. This does not imply independent or immediately accurate samples. A chain stuck in one plausible region can give misleading frequencies for a long time.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 16.

Lecture 17: Decision Networks and Value of Information

17.1 Decisions and expected utility

A decision network augments a Bayes net with decision nodes and a utility node. Chance nodes describe uncertainty, a decision node specifies an action, and utility depends on the action and relevant outcomes. For current evidence e, evaluate each action by

$$EU(a \mid e) = \sum_s P(s \mid e)U(a, s), \qquad MEU(e) = \max_a EU(a \mid e),$$

in the setting where the action does not change the chance distribution. More general action-dependent outcomes use the corresponding model for each action. Choose the maximizing action after inferring the utility parents.

For the umbrella example, utilities for leaving it are 100 in sun and 0 in rain; taking it gives 20 in sun and 70 in rain. With sun probability 0.7, leaving has value 70 and taking has value 35. A bad forecast that changes sun probability to 0.34 changes these to 34 and 53, making taking the umbrella preferable.

The shape of each node describes its role. A chance node represents a random quantity, a rectangular decision node represents a choice, and a utility node contains a numerical preference rather than a probability distribution. Utility need not sum to one over outcomes.

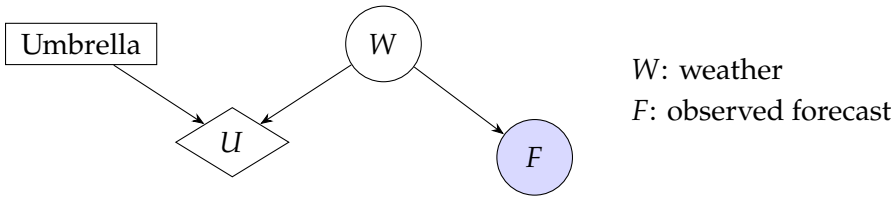


Figure 18: Evidence changes beliefs about weather, while the decision changes utility.

To evaluate taking the umbrella, hold that action fixed and average its two utility entries using the current weather probabilities. To evaluate leaving it, use the same probabilities with the other utility entries. The two actions need not have separate probability models if neither changes the weather. Changing a decision is not the same as observing a random node.

The bad-forecast numbers show the complete calculation: $EU(\text{leave} \mid \text{bad}) = 0.34(100) + 0.66(0) = 34$ and $EU(\text{take} \mid \text{bad}) = 0.34(20) + 0.66(70) = 53$. Inference supplies the distribution; optimization uses that distribution to select the action.

17.2 Value of information

Before observing a variable Z, its value is uncertain. The VALUE OF PERFECT INFORMATION about that variable is

$$VPI(Z \mid e) = \sum_z P(z \mid e) MEU(e, z) - MEU(e).$$

The maximum is taken separately after each possible observation: information is useful because the action can adapt. In the oil example, exactly one of two equally likely locations holds oil worth k. Choosing without a survey has value $k/2$; learning its location gives value k, so the information is worth $k/2$ in the same utility units.

VPI is nonnegative because the agent can ignore the observation and keep the old action. It can be zero even for a surprising observation if no action changes. Information values are not generally additive: two observations can overlap or become useful only together. Total value of observing a set is independent of its order, although the conditional value of the second observation depends on what is already known. Information acquisition is worthwhile when its expected benefit exceeds its cost in utility.

A noisy report is represented by observing a noisy sensor node, not by pretending to observe the underlying state. If the new variable is conditionally independent of the relevant uncertain utility parents given current evidence and every candidate action, it cannot improve the decision.

The timing of the maximum is essential. Without information, choose one action before learning Z. With information, choose a potentially different action for each observed value. In general,

$$\sum_z P(z \mid e) \max_a EU(a \mid e, z) \;\geq\; \max_a \sum_z P(z \mid e) EU(a \mid e, z).$$

The right side equals acting before seeing Z; the left side allows adaptation. This inequality gives the nonnegativity of information value.

For an illustrative noisy oil report that identifies the correct location with probability 0.8, following the report succeeds with probability 0.8 and has expected utility $0.8k$. Without it, the best value is $0.5k$, so this report is worth $0.3k$. Learning the true location itself is worth $0.5k$. Perfect observation of a noisy report is different from perfect observation of the underlying location.

Two observations can be redundant: after the exact oil location is known, a second report about the same location has no value. They can also be complementary if neither alone changes the best action but their combination does. This is why separate information values cannot simply be added.

A variable can have high uncertainty and still zero decision value. If both possible soups would lead to ordering a sandwich, learning which soup is available changes beliefs but not the action. Information is valuable relative to a decision and utility function, not merely because it reduces uncertainty.

17.3 Partial observability

A PARTIALLY OBSERVABLE MDP adds an observation model to an MDP. The agent maintains a BELIEF STATE, a probability distribution over physical states. An action changes the physical state; an observation updates the belief. Planning can therefore be viewed as an MDP over beliefs, though the belief space is typically continuous and much larger.

In static Ghostbusters, beliefs are determined by accumulated sensor evidence. A truncated expectimax tree can compare acting immediately against sensing once and then acting. This produces a VPI-based sensing rule, while longer planning can account for sequences of information-gathering actions.

A belief state keeps track of what the agent knows, not what the physical state actually is. Two agents at the same physical location can choose different sensing actions because their information differs. After an action and observation, Bayes' rule produces a new belief, and planning continues from that belief.

A one-step VPI policy considers only sensing now and then acting. It can miss a pair of measurements that are useful together or the future benefit of traveling to a better sensing position. Full belief-space planning accounts for those possibilities, at a much greater computational cost.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 17.

Lecture 18: Hidden Markov Models

18.1 Markov chains and stationary distributions

A first-order Markov chain represents a sequence of states with $P(X_{t+1} \mid X_{1:t}) = P(X_{t+1} \mid X_t)$. A stationary transition model uses the same conditional table at every time. Starting from a distribution b_t, prediction pushes probability through the transitions:

$$b_{t+1}(x') = \sum_x P(x' \mid x) b_t(x).$$

A stationary distribution satisfies $\pi(x') = \sum_x P(x' \mid x)\pi(x)$. It need not be the limit from every initial distribution for an arbitrary chain; convergence requires additional conditions. In the weather example, $P(\text{sun} \mid \text{sun}) = 0.9$ and $P(\text{sun} \mid \text{rain}) = 0.3$ give $\pi(\text{sun}) = 0.9\pi(\text{sun}) + 0.3\pi(\text{rain})$, hence $\pi(\text{sun}) = 3/4$ and $\pi(\text{rain}) = 1/4$.

PageRank uses a random walk on pages with occasional random jumps, assigning larger stationary mass to more reachable pages. Gibbs sampling similarly constructs a chain over complete assignments whose stationary distribution is the desired posterior.

The prediction equation sums over ways to arrive at the same next state. If the current sun probability is 1, the next sun probability is 0.9. After another step it is $0.9(0.9) + 0.3(0.1) = 0.84$: both a sunny current state that stays sunny and a rainy current state that changes to sun contribute.

A stationary *model* and a stationary *distribution* are different. The model is stationary when its transition probabilities do not change with time. The current distribution can still evolve. A stationary distribution is a special input distribution left unchanged by that model. For the example, normalization plus $0.1\pi(\text{sun}) = 0.3\pi(\text{rain})$ gives the $3 : 1$ ratio.

One should not assume every chain forgets its start. Two absorbing states can retain their initial masses forever. A deterministic two-state alternation can oscillate instead of converging. These cases explain why the fixed-point equation alone does not prove convergence from all starts.

18.2 Hidden states and observations

A hidden Markov model (HMM) adds observations E_t emitted from hidden states X_t. It specifies an initial state distribution, transitions $P(X_t \mid X_{t-1})$, and emissions $P(E_t \mid X_t)$:

$$P(x_{1:T}, e_{1:T}) = P(x_1)P(e_1 \mid x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})P(e_t \mid x_t).$$

The hidden process is Markov, and the current observation is independent of the other variables given the current state. Observations are not generally independent without conditioning on states.

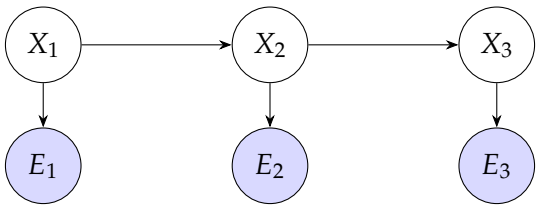


Figure 19: Hidden dynamics and observed emissions in an HMM.

For weather tracking, the state may be whether it rains, while the observed variable is whether someone carries an umbrella. An umbrella is evidence of rain, not rain itself. A person can carry one on a sunny day, so the posterior need not become certain after seeing it.

The two independence assumptions have separate jobs. The transition model lets prediction summarize all past observations by the current belief. The emission model lets an observation update use only the candidate current state. Without these assumptions, the belief needed for an exact online update might have to retain more history.

18.3 Filtering and the forward algorithm

Filtering maintains $b_t(x) = P(X_t = x \mid e_{1:t})$. Prediction and observation updates are

$$\bar{b}_{t+1}(x') = \sum_x P(x' \mid x) b_t(x), \qquad b_{t+1}(x') = \alpha P(e_{t+1} \mid x') \bar{b}_{t+1}(x').$$

The first equation follows by marginalizing the previous state and using the Markov property. The second is Bayes' rule and emission independence. Initialize by weighting the initial prior with $P(e_1 \mid x_1)$. Predictions often spread belief; informative observations often concentrate it, although an individual observation need not reduce entropy.

The unnormalized forward algorithm instead carries $f_t(x) = P(x_t, e_{1:t})$:

$$f_{t+1}(x') = P(e_{t+1} \mid x') \sum_x P(x' \mid x) f_t(x).$$

Normalize to recover the filtering distribution; scaling each step avoids numerical underflow. With $|S|$ states, a dense update costs $O(|S|^2)$ and stores only the current vector. In robot localization, states are positions and observations are noisy sensor readings; observations disambiguate positions that the motion model alone cannot distinguish.

18.4 A complete weather-filtering calculation

Use $P(r_{t+1} \mid r_t) = 0.7$, $P(r_{t+1} \mid \neg r_t) = 0.3$, and umbrella likelihoods $P(u_t \mid r_t) = 0.9$, $P(u_t \mid \neg r_t) = 0.2$. Start with rain probability $1/2$ before the first observation.

After observing an umbrella, the unnormalized rain and sun masses are $0.9(0.5) = 0.45$ and $0.2(0.5) = 0.1$. Dividing by 0.55 gives $P(r_1 \mid u_1) = 9/11$. Predicting the next day gives

$$\bar{b}_2(r) = 0.7\frac{9}{11} + 0.3\frac{2}{11} = \frac{69}{110}, \qquad \bar{b}_2(\neg r) = \frac{41}{110}.$$

Another umbrella weights these by 0.9 and 0.2. The new posterior is

$$b_2(r) = \frac{0.9(69/110)}{0.9(69/110) + 0.2(41/110)} = \frac{621}{703} \approx 0.883.$$

Prediction moved the belief toward the weather dynamics; the new observation then moved it back toward rain.

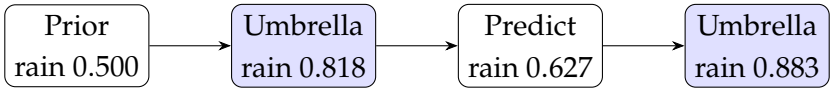


Figure 20: Filtering alternates motion prediction and likelihood reweighting.

Normalization at each step retains a usable belief distribution. If only relative masses are needed for later filtering, any common positive scaling factor can be removed. If the probability of the whole observation sequence is needed, those scaling factors must be retained. Discarding them preserves the posterior shape but loses the sequence likelihood.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 18.

Lecture 19: Particle Filtering and Applications of HMMs

19.1 Particles as a belief representation

Exact filtering is impractical when the state space is enormous or continuous. A particle filter represents a belief with N sampled states $x^{(1)}, \ldots, x^{(N)}$ rather than a table over all possible states. In a discrete space, the fraction equal to x approximates its probability. Store the particles themselves; a table over every possible state defeats the space saving.

One filtering step has three operations:

1. Predict each particle independently by drawing $\tilde{x}^{(i)} \sim P(X_{t+1} \mid x^{(i)})$.
2. Weight the predicted particles by the fixed observation: $w_i = P(e_{t+1} \mid \tilde{x}^{(i)})$.
3. Draw N new particles with replacement from the predicted particles, with selection probability $w_i / \sum_j w_j$.

The resulting particles again have equal weights. Prediction approximates the transition sum; weighting and resampling approximate the observation update. Resampling replicates plausible states and removes implausible ones, but introduces sampling noise. If all weights are zero, normalization is undefined and the representation must be recovered rather than silently continuing. More particles help, but difficult beliefs may require many.

With efficient transition draws, likelihood evaluation, and resampling, work is linear in N per step. In localization the map is known and the robot's pose is uncertain. In simultaneous localization and mapping (SLAM), both pose and map are unknown, so the state is substantially larger.

Example: For an illustrative four-particle prediction, suppose the states are $[A, A, B, C]$ and the observation likelihoods are $P(e \mid A) = 0.8$, $P(e \mid B) = 0.2$, and $P(e \mid C) = 0$. The individual weights are $[0.8, 0.8, 0.2, 0]$, totaling 1.8. The resampling probability of selecting a particle at A is $1.6/1.8 = 8/9$, at B it is $1/9$, and at C it is zero. A possible resampled collection is $[A, A, A, B]$; another draw could produce four copies of A.

Repeated particles are not an error. Their multiplicities represent probability mass. Replacing the list by its set of distinct states would discard that information. Conversely, particle filtering does not say that every unrepresented state is impossible in the true model: it may simply have been missed by the finite sample.

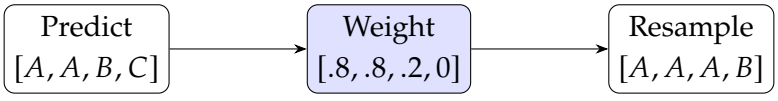


Figure 21: Resampling turns unequal weights back into an equally weighted particle list.

Algorithm 19.1: One bootstrap particle-filter step

Input: Particles $x^{(1)}, \ldots, x^{(N)}$ and observation e
for $i = 1, \ldots, N$ **do**
  Sample $\tilde{x}^{(i)}$ from the transition model given $x^{(i)}$
  Set $w_i = P(e \mid \tilde{x}^{(i)})$
if $\sum_i w_i = 0$ **then**
  return a failure indication for this particle representation
Normalize the weights
Draw N indices with replacement using the normalized weights
return the predicted particles at those indices, now with equal weights

Resampling focuses effort where the latest data says the state is plausible. It can also lose diversity. Once every particle has left a region, a future observation favoring that region may be hard to handle unless the transition model can move particles back there.

19.2 Dynamic Bayes nets

A DYNAMIC BAYES NET repeats a structured collection of variables at each time, with dependencies within a slice and from the preceding slice. It generalizes an HMM whose single state variable encodes the whole slice.

Exact inference unrolls the network and eliminates old variables, retaining factors needed for the present. These factors may become large. For a particle filter, each particle must be a complete joint assignment to the slice, such as both ghosts' positions. Sample its successor from the joint transition model and weight it by the observation likelihood. When emissions are conditionally independent, multiply their likelihood factors. Sampling each tracked entity independently would lose dependencies represented by the joint model.

19.3 Most likely state sequences

Filtering asks for a marginal distribution at the current time. A MOST LIKELY EXPLANATION instead asks for the single sequence $\arg\max_{x_{1:T}} P(x_{1:T} \mid e_{1:T})$. The evidence normalizer does not depend on the sequence, so maximize the joint.

The VITERBI ALGORITHM replaces forward sums over paths by maxima:

$$v_1(x) = P(x)P(e_1 \mid x), \qquad v_{t+1}(x') = P(e_{t+1} \mid x') \max_x [P(x' \mid x) v_t(x)].$$

Store the maximizing predecessor for each state and time. Choose the best final state, then trace back these pointers. A state trellis makes the distinction clear: forward inference sums the weights of all paths reaching a node, whereas Viterbi retains only the best path. Choosing the most likely state separately at each time need not give the most likely valid sequence.

For Viterbi, $v_t(x)$ is the probability of the *best complete prefix* ending in x, including observations through time t. To extend to x', compare every predecessor's best prefix times the transition probability. The emission probability for x' then multiplies the winning result.

A small illustration shows why marginal modes are insufficient. Suppose the only two-state paths with nonzero posterior masses are $(A, B) : 0.4$, $(B, A) : 0.35$, and $(B, B) : 0.25$. The most probable first state is B and the most probable second state is B, but the path (B, B) has probability only 0.25. The most likely path is (A, B) with probability 0.4.

Backpointers preserve the choices that made the best prefix. Keeping only the final scores would identify the best last state but lose the earlier states needed for the explanation. In a dense model, both forward and Viterbi updates cost $O(|S|^2)$ per time step; storing a complete Viterbi trace requires predecessor information over time.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 19.

Lecture 20: Machine Learning: Naive Bayes

20.1 Classification and the model

Supervised classification learns from labeled inputs (x, y) to predict labels for new inputs. Features encode properties such as word occurrences, sender identity, or image pixels. A naive Bayes classifier assumes that features are conditionally independent given the label:

$$P(y, f_1, \ldots, f_n) = P(y) \prod_i P(f_i \mid y), \qquad \hat{y} = \arg\max_y P(y) \prod_i P(f_i \mid y).$$

Normalize the class scores for a posterior distribution. Conditional independence is a modeling assumption, not a claim that features are unconditionally independent. The model has only linearly many local feature tables rather than an exponential table over all features.

For text, a bag-of-words model makes word positions conditionally independent and identically distributed given the class, tying $P(W_i \mid Y)$ across positions. The score is insensitive to order. Compute log scores, $\log P(y) + \sum_i \log P(w_i \mid y)$, to avoid underflow.

The prediction rule comes from Bayes' rule: $P(y \mid f) \propto P(y, f)$. Substitute the naive Bayes factorization, then compare class scores. The evidence probability is common to every candidate class and can be omitted when only the maximizing label is needed. It must be restored by normalization if posterior probabilities are requested.

For a binary pixel, both an on value and an off value are evidence. Ignoring off pixels changes the model. In a word-position model, by contrast, the observations are the words present at the positions. A binary word-occurrence model and a repeated-token bag-of-words model are different representations and need different counts.

Conditional independence permits strong simplification but can exaggerate confidence when several features repeat the same evidence. Two nearly identical word-pattern features might each multiply in a similar likelihood ratio, effectively counting the clue twice.

20.2 Parameter estimation and smoothing

Maximum likelihood estimates categorical probabilities by relative frequencies. For a feature value v in class y,

$$\widehat{P}(F_i = v \mid y) = \frac{N(F_i = v, y)}{N(y)}.$$

An unseen event then receives probability zero, eliminating a class no matter how strong the other evidence is. Laplace smoothing adds $k > 0$ pseudo-counts to each of the d_i possible feature values:

$$\widehat{P}(F_i = v \mid y) = \frac{N(F_i = v, y) + k}{N(y) + kd_i}.$$

Smooth each conditioning context separately. In a word-position model, count tokens within each class and use the vocabulary size in the denominator. Larger k pulls toward a uniform distribution. Linear interpolation is an alternative: mix the empirical conditional with an overall empirical distribution, reducing reliance on sparse conditional counts.

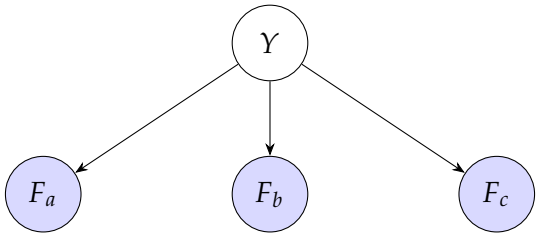
Example: Consider an illustrative one-feature spam model. Among 4 spam emails, 3 contain a chosen word; among 6 ham emails, 1 contains it. With add-one smoothing, $P(\text{word present} \mid \text{spam}) = (3+1)/(4+2) = 2/3$ and $P(\text{word present} \mid \text{ham}) = (1+1)/(6+2) = 1/4$. Using unsmoothed class priors $0.4, 0.6$, the scores for a word-containing email are $0.4(2/3) = 4/15$ and $0.6(1/4) = 3/20$. The spam posterior is

$$\frac{4/15}{4/15 + 3/20} = \frac{16}{25} = 0.64.$$

The choice to smooth the prior separately is independent of smoothing the feature tables.

The denominator adds k once for *every possible value*. A binary feature adds $2k$ to its class count, not merely k. For a vocabulary of size V, a token distribution adds Vk to the token total. The numerator and denominator must describe the same kind of observation.

When using logarithms, sum the log prior and feature log likelihoods. To recover normalized probabilities, subtract the largest log score before exponentiating. This common shift leaves relative scores unchanged while avoiding extremely small raw products.



Feature values are observed.
Compare a class score for each possible Y.

Figure 22: Naive Bayes connects each observed feature to the unknown label.

20.3 Generalization and evaluation

Training learns parameters. A separate held-out set selects hyperparameters such as smoothing strength; the test set estimates final performance and must not guide repeated tuning. Overfitting occurs when training peculiarities or noise are fitted at the expense of future prediction. More data,

simpler representations, and regularization can help.

Compare against a baseline such as always predicting the most frequent training label. High accuracy on an imbalanced problem can conceal a weak classifier. Inspecting errors can reveal missing features, such as whether the sender is known or whether a displayed link matches its destination.

A probabilistic classifier's top posterior is a confidence estimate, but the model can be wrong about that confidence. Calibration asks whether predictions made with a given confidence are correct at the corresponding rate; a high numerical posterior alone is no guarantee of reliability.

A parameter is estimated within a chosen model, such as a feature probability. A hyperparameter chooses how that estimation is done, such as smoothing strength. Trying many smoothing values on the test set would gradually fit the evaluation procedure to that same test set. Held-out tuning protects the final test from this feedback.

Accuracy also needs context. If most messages are ham, always saying ham can obtain a high score while missing every spam message. Compare with that baseline and inspect which kinds of errors remain. A confident mistake can reveal a stronger modeling problem than a near-tie mistake.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 20.

Lecture 21: Machine Learning: Perceptrons and Logistic Regression

21.1 Linear scores and perceptron updates

A feature vector $f(x)$ maps an input to numbers. A linear classifier scores it by $z = w^\top f(x)$ and predicts the sign. A constant feature represents a bias, allowing the separating hyperplane to move away from the origin.

The binary PERCEPTRON starts with zero weights. For each example with gold label $y^* \in \{-1, +1\}$, predict using the current score. On a mistake, update $w \leftarrow w + y^* f(x)$; otherwise leave it unchanged. The update raises the gold label's signed score on that example.

For multiple classes, keep a vector w_y for each class and predict $\hat{y} = \arg\max_y w_y^\top f(x)$. On a mistake,

$$w_{y^*} \leftarrow w_{y^*} + f(x), \qquad w_{\hat{y}} \leftarrow w_{\hat{y}} - f(x).$$

Only the correct and incorrectly predicted class vectors change. The relative score of the correct class increases on the mistaken input.

If the binary training set is linearly separable, perceptron training converges after finitely many mistakes; the margin affects the mistake bound. Nonseparable data can make weights keep changing. Averaging successive weight vectors can stabilize predictions. A separating boundary need not generalize well, and repeated passes can overfit; held-out performance helps choose when to stop.

The weights tell how a feature changes the score while other features are held fixed. A positive weight raises the positive-class score; a negative weight lowers it. The bias is the weight of an always-one feature. For the rule $4f_{\text{free}} + 2f_{\text{money}} - 3 > 0$, the boundary is $4f_{\text{free}} + 2f_{\text{money}} = 3$.

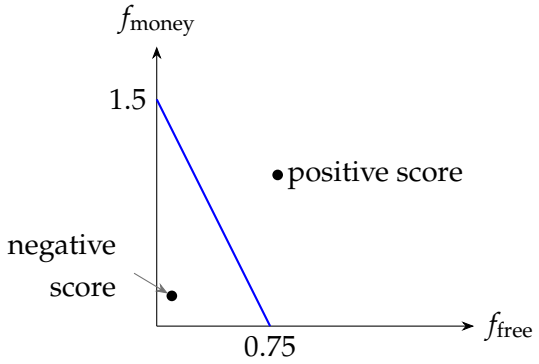


Figure 23: A linear score separates feature vectors by a straight boundary.

For a binary update, take an illustrative mistaken positive example $f = (2, 1)$ and current weights

$w = (-1, 0)$. The score is -2, so the prediction is negative. Adding the positive example gives $w' = (1, 1)$ and changes this example's score to 3. In general, the gold signed score changes by $y^*(w + y^* f)^\top f - y^* w^\top f = \|f\|^2$. This explains the direction of the update, but an update can also change scores on other examples.

Algorithm 21.1: Multiclass perceptron training

```
Initialize each class vector w_y = 0
for each training pass do
    for each example (x, y*) do
        Predict ŷ ∈ arg max_y w_y^⊤ f(x) using a fixed tie rule
        if ŷ ≠ y* then
            w_{y*} ← w_{y*} + f(x)
            w_ŷ ← w_ŷ − f(x)
```

The separability assumption means that one linear boundary can classify every training example correctly, with ties handled consistently. A positive margin means the examples lie a nonzero distance from that boundary after the separating direction is normalized. A narrow margin allows more mistakes before convergence. If labels conflict or the needed boundary is nonlinear in the chosen features, repeated updates need not settle. More passes are not a substitute for an adequate representation.

21.2 Logistic probabilities

Perceptron scores are not probabilities. Binary LOGISTIC REGRESSION uses the sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad P(y = +1 \mid x; w) = \sigma(w^\top f(x)), \quad P(y = -1 \mid x; w) = 1 - \sigma(w^\top f(x)).$$

The sign still determines the most likely class, but the score's magnitude controls confidence. Learn weights by maximizing conditional log likelihood:

$$\ell(w) = \sum_i \log P(y^{(i)} \mid x^{(i)}; w).$$

This objective uses degrees of confidence rather than changing weights only on mistakes. It can learn probabilistic predictions even when no linear boundary perfectly separates the labels.

The sigmoid maps score 0 to probability $1/2$, score $\log 3$ to $3/4$, and score $-\log 3$ to $1/4$. Its output is always between zero and one, but that alone does not prove the probabilities are calibrated on real data.

For a positive training label, the log-likelihood contribution is $\log p$; for a negative label it is $\log(1-p)$. A confident wrong prediction receives a much larger penalty than an uncertain one. For example, a positive label with $p = 0.01$ contributes about -4.605, while $p = 0.5$ contributes about -0.693. This gives a graded training signal even when two models make the same hard prediction.

Separating data perfectly does not by itself give finite optimal logistic weights without further constraints or regularization: enlarging separating scores can keep increasing the unregularized likelihood. Validation and regularization concern how the probabilities generalize, not only whether training labels fall on the correct side.

21.3 Multiclass logistic regression

For class scores $z_y = w_y^\top f(x)$, SOFTMAX gives

$$P(y \mid x; w) = \frac{e^{z_y}}{\sum_{y'} e^{z_{y'}}}.$$

The largest score remains the most probable label, and the probabilities sum to one. Optimize the sum of the gold labels' log probabilities on training data. All class scores enter the normalizer, so increasing one class's score changes the probabilities of the others. Subtracting a common constant from the scores preserves these probabilities and permits stable computation.

For an illustrative three-class score vector $(\log 2, 0, 0)$, exponentiation gives weights $(2, 1, 1)$, so softmax produces $(1/2, 1/4, 1/4)$. These are relative probabilities: increasing one score changes the shared denominator. Subtracting $\max_y z_y$ before exponentiating leaves the ratios unchanged and prevents overflow from unnecessarily large exponents.

The most likely class still has the highest linear score. Turning scores into probabilities changes the training objective and the confidence interpretation, not the shape of the hard decision boundaries in the same feature space.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 21.

Lecture 22: Machine Learning: Optimization and Neural Networks

22.1 Gradient optimization

To maximize a differentiable objective $g(w)$, gradient ascent uses $w \leftarrow w + \alpha \nabla g(w)$. The first-order approximation $g(w+\Delta) \approx g(w) + \nabla g(w)^\top \Delta$ explains the direction: among steps with fixed Euclidean length, the gradient has greatest predicted increase. The learning rate α controls distance; an excessively large step can overshoot, while a small step can make progress slow.

For conditional log likelihood $\ell(w) = \sum_i \log P(y^{(i)} \mid x^{(i)}; w)$, batch ascent sums gradients over the entire training set before updating. Stochastic ascent updates from a randomly selected example, and mini-batch ascent uses a small random subset, allowing parallel computation with more frequent updates than a full batch. Their gradient scaling and learning rate must be used consistently.

For a binary label $t \in \{0, 1\}$ and $p = \sigma(w^\top f)$, differentiating $t \log p + (1-t)\log(1-p)$ gives $(t-p)f$. This displays the training signal: raise scores where the observed label exceeds the predicted probability.

The gradient is a vector of partial derivatives, one for each parameter. For a unit-length proposed direction d, the predicted change is $\alpha \nabla g(w)^\top d$. By the Cauchy–Schwarz inequality this is at most $\alpha \|\nabla g(w)\|$, attained in the gradient direction. This statement is local: it does not guarantee that a long step keeps increasing the actual function.

For the illustrative objective $g(w) = -(w-2)^2$, the derivative is $4 - 2w$. Starting at $w = 0$ with step size $1/4$ gives $w = 1$, then 1.5, then 1.75. The steps shrink as the slope shrinks. A very large step could jump back and forth across the optimum instead.

The logistic gradient can be derived rather than memorized. Since $dp/dz = p(1-p)$ and $z = w^\top f$,

$$\frac{\partial \ell}{\partial z} = \left(\frac{t}{p} - \frac{1-t}{1-p}\right) p(1-p) = t - p, \qquad \frac{\partial \ell}{\partial w_i} = (t-p) f_i.$$

For softmax with gold label y^*, the corresponding class-vector gradient is $(\mathbf{1}\{y = y^*\} - P(y \mid x)) f(x)$. The correct class is pushed up, while competing classes are pushed down in proportion to their current probabilities.

Stochastic gradients are noisy estimates of a full-data direction. They can make useful progress before an expensive full pass is finished, but the objective need not increase on every individual step. Mini-batches reduce some noise and suit parallel computation.

22.2 Learning representations

Logistic regression applies a learned linear score to fixed features. A NEURAL NETWORK learns intermediate features as well:

$$h^{(0)} = x, \qquad h^{(k)} = g(W^{(k)}h^{(k-1)} + b^{(k)}), \qquad P(y \mid x) = \text{softmax}(W^{\text{out}}h^{(L)} + b^{\text{out}})_y.$$

Nonlinear activations such as sigmoid, hyperbolic tangent, or $\text{ReLU}(z) = \max(0, z)$ permit nonlinear input-output functions. Without nonlinearities, composing affine layers would remain affine.

Training again maximizes conditional log likelihood, now over all layers' weights. Large networks can fit complex structure and can also memorize training data. Early stopping uses held-out performance to limit overfitting; training accuracy alone does not identify the best stopping point.

A hidden unit first computes a weighted combination of inputs, then applies a nonlinearity. A later layer combines those learned intermediate values. Thus a feature does not need to be supplied as a handcrafted rule before training; its parameters can change to make the final prediction better.

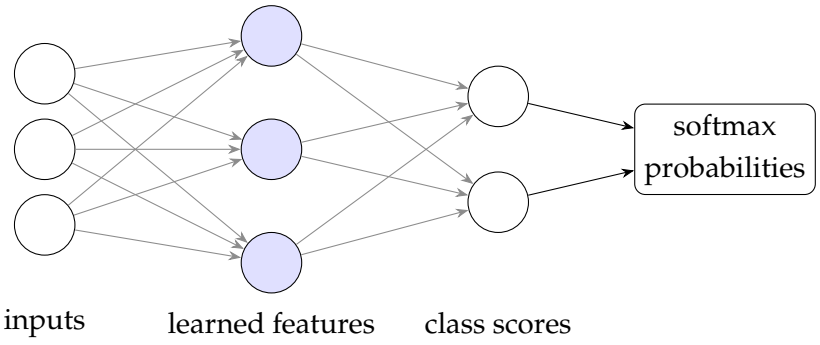


Figure 24: Hidden layers learn features that feed the final classifier.

For ReLU, negative inputs become zero and positive inputs pass through. Sigmoid compresses large positive and negative inputs toward its extremes. The nonlinearity changes which input patterns a layer can represent. Merely adding more linear layers would not create nonlinear boundaries, because a product of linear maps is still linear.

Depth and width control expressive capacity differently, but neither removes the need for data and optimization. An approximation theorem says some weights exist for a sufficiently large model; it does not give those weights or show that a finite dataset identifies them.

22.3 Approximation and automatic differentiation

With suitable nonlinear activations, a network with one sufficiently wide hidden layer can approximate continuous functions on a compact domain to arbitrary accuracy. This is an expressiveness result; it does not guarantee that training finds the weights, that the network is small, or that it

generalizes from finite data.

A network is a composition of simple operations, so the chain rule computes its derivatives. Automatic differentiation records intermediate values on a forward pass and propagates derivative information backward. This backpropagation computes gradients with respect to many parameters at a cost comparable in order to evaluating the network, rather than separately perturbing every weight. Learned features support tasks such as object recognition, speech recognition, translation, and captioning.

For a scalar chain-rule illustration, let $u = wx$, $h = \sigma(u)$, and $z = vh$. A forward pass computes and stores u, h, z. A derivative with respect to w follows the chain:

$$\frac{\partial z}{\partial w} = \frac{\partial z}{\partial h}\frac{\partial h}{\partial u}\frac{\partial u}{\partial w} = v\, h(1-h)x.$$

In a large network, many parameters share intermediate computations. The backward pass reuses those stored results rather than recomputing the entire model separately for each parameter. Training still needs a learning rate, data selection, and a stopping rule; automatic differentiation supplies derivatives, not those other decisions.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 22.

Lecture 23: Machine Learning: Neural Networks II and Decision Trees

23.1 Induction, consistency, and simplicity

Learning chooses a hypothesis h from a hypothesis space H using examples of an unknown target function. Classification predicts discrete labels; regression predicts real values. A hypothesis consistent with all observed examples can still be wrong elsewhere. Expressive models, including neural networks, can represent more targets but also fit more accidental patterns.

The tradeoff between consistency and simplicity can be controlled by reducing the hypothesis space, selecting features, adding modeling assumptions, or regularizing selection within a large space. These choices balance underfitting against sensitivity to the sample, often described as bias and variance. Held-out performance evaluates that balance.

23.2 Decision trees and expressiveness

A decision tree tests an attribute at each internal node, follows the matching branch, and predicts at a leaf. Each root-to-leaf path represents a conjunction of tests. A feature can have different effects in different branches, automatically capturing interactions that a linear classifier would need explicit conjunction features to represent.

For finitely many discrete attributes, a sufficiently large tree can represent any function by distinguishing complete assignments. With n Boolean attributes, there are 2^{2^n} Boolean functions, so unrestricted expressiveness is enormous. The practical aim is a small useful tree rather than a table memorizing every assignment.

A tree can reuse a feature in different contexts because each branch represents a different subset of examples. A linear classifier on raw features adds fixed contributions. To express that one feature matters only when another is present, it needs an appropriate interaction feature. A tree obtains such conjunctions through its paths.

Expressing a function and finding it from examples are separate problems. The full truth table of n Boolean attributes has 2^n rows. A tree that distinguishes all rows can represent any labeling, but the number of possible labelings is 2^{2^n}. A learner needs a preference for some hypotheses over others; the training examples alone may leave many possibilities.

23.3 Information gain and recursive learning

Grow a tree recursively by choosing an informative attribute, splitting examples by its values, and repeating within each subset. A pure subset can become a leaf. When no useful split remains, use a label based on the examples reaching the leaf.

For label proportions $p_1, \dots, p_k$, ENTROPY is

$$H(Y) = -\sum_j p_j \log_2 p_j, \qquad 0 \log_2 0 := 0.$$

It measures average information in the label: a fair binary label has one bit, while a certain label has zero. The INFORMATION GAIN of attribute A is

$$IG(A) = H(Y) - \sum_a \frac{N_a}{N} H(Y \mid A = a).$$

Use subset frequencies N_a/N to weight the post-split entropies. Choose the largest gain and recurse. This greedy rule need not find the globally best tree: useful interactions can be invisible at an individual split.

For a worked split, suppose eight examples have four positive and four negative labels. Their entropy is 1 bit. Attribute A splits them into two equal groups, with label counts $(3, 1)$ and $(1, 3)$. Each group's entropy is

$$-\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} \approx 0.8113.$$

The weighted post-split entropy is also 0.8113, so the gain is about 0.1887 bits. A different attribute separating the four positives from the four negatives has zero entropy in both children and gain 1 bit. This explains why pure children are preferred.

The weights N_a/N are important. A split that isolates one easy example while leaving a large mixed group should not be judged by an unweighted average of its children. Information gain uses the label uncertainty of a randomly selected training example after its branch is known.

Algorithm 23.1: Recursive decision-tree learning

Input: Examples E, remaining attributes A, and a fallback label
if E is empty **then**
  return a leaf with the fallback label
if all labels in E agree **then**
  return a leaf with that label
if A is empty or a stopping rule applies **then**
  return a leaf with the majority label in E
Choose an attribute X with largest information gain
Create a decision node testing X
for each value v of X **do**
  Recursively learn a child from $\{e \in E : X(e) = v\}$ and $A \setminus \{X\}$, using the current majority label as fallback
return the decision node

23.4 Overfitting and pruning

Small subsets can appear pure by chance. Continuing to split them can improve training accuracy while harming generalization. One approach builds a tree and prunes bottom-up, removing splits whose apparent predictive relationship is not sufficiently supported, for example under a chi-squared significance criterion. Later structure can redeem a split that looks uninformative by itself, as with XOR, so premature stopping can miss interactions.

A pruning threshold is a regularization hyperparameter chosen on held-out data. A maximum-depth restriction instead directly limits the hypothesis space. Smaller trees can underfit, while larger trees can fit noise; the test set is reserved for evaluating the final choice.

XOR is a useful warning about greedy stopping. Let the label be one when two Boolean attributes differ and zero when they agree. At the root, either attribute alone leaves a balanced pair in each branch, so its information gain is zero. Splitting on both attributes nevertheless represents the function perfectly.

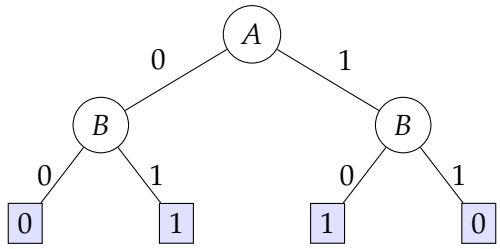


Figure 25: A two-level tree captures an interaction invisible to either root split alone.

Pruning asks whether a more complicated subtree is justified by evidence, not whether every split improved training fit. Tiny sample counts often produce apparent patterns by accident. A significance-based rule compares observed label counts with what would be expected if the attribute and label were unrelated. Its threshold is tuned on held-out data.

A simpler tree is easier to estimate from limited data, but simplicity is not always correctness. An overly aggressive pruning rule can remove a real interaction. The goal is to select a complexity supported by data, rather than to maximize or minimize tree size for its own sake.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 23.

Lecture 24: Advanced Applications: Robotics, Vision, and Control

24.1 Combining search and learning

Large applications combine the course's methods. In Go, exhaustive minimax is infeasible because of branching and depth. A policy network narrows plausible moves; a value network estimates positions without reaching the end. Supervised learning from expert play, reinforcement learning through self-play, and Monte Carlo rollouts contribute complementary information.

A policy network and a value network answer different questions. The policy suggests which moves to consider; the value estimates how promising the position is if play continues. A rollout instead samples a continuation and uses its outcome as evidence about the position. Search combines these signals to spend computation where it is most useful.

Learning from expert moves provides an initial policy, while self-play can generate experience beyond those examples. The learned models guide search; they do not make the game's legal moves or terminal rewards unnecessary. The application therefore connects learned estimates with an explicit decision problem.

24.2 Helicopter estimation and control

A helicopter's state includes position, orientation, and their rates of change. Cameras and inertial sensors provide noisy measurements; a dynamics model predicts motion under control inputs:

$$s_{t+1} = f(s_t, a_t) + w_t.$$

Tracking combines prediction with observation updates, as in an HMM. Control chooses inputs affecting the main rotor and tail rotor. A hovering objective penalizes deviation from a target pose, but complex maneuvers also require specifying a useful target trajectory.

Demonstrations can be observations of an underlying desired trajectory. Their speeds differ, so directly averaging equal time indices blurs the maneuver. Probabilistic alignment combines a dynamics model with temporal alignment and state estimation to recover a cleaner latent trajectory. A controller can then track the learned maneuver. This connects imitation, sequence inference, and control rather than treating demonstrations as independent input-output examples.

The estimation and control problems form a feedback loop. First predict the next state using the previous estimate and the control just applied. Then compare new sensor readings with what candidate states would predict, and update the estimate. The controller chooses its next input using that updated information.

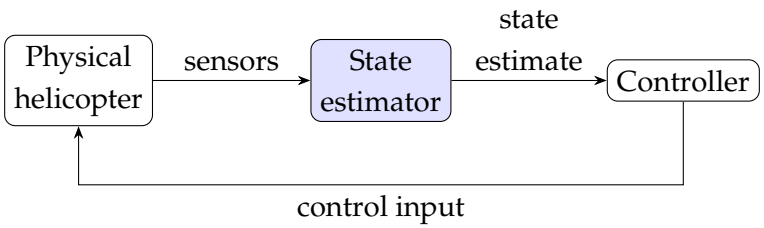


Figure 26: Control uses an estimated state and receives new feedback after acting.

An inertial sensor measures motion-related quantities, while vision provides external position information. Each is imperfect. Combining them through a dynamics and observation model is more useful than treating every reading as an exact state. The same position can require different controls when velocities or orientations differ, explaining why the state includes these quantities.

Demonstration alignment solves a different uncertainty: which moments in different demonstrations correspond to the same part of a maneuver? A slow demonstration and a fast one should not be averaged at identical clock times. A latent trajectory plus a timing alignment can pool evidence about the maneuver's actual shape.

24.3 Locomotion and rewards

Continuous-control policies can map joint angles and velocities through a neural network to joint torques. Learning in simulation permits repeated trials, while the representation must support coordinated motion. Hierarchical control separates a high-level choice, such as where to place a foot, from low-level motor commands that execute it.

For terrain traversal, a reward of the form $R(s) = w^\top f(s)$ can express preferences over terrain properties. Demonstrated paths help infer reward weights; planning with those weights then chooses paths across a new height map. Learning the objective can matter as much as optimizing a fixed one.

Learning a reward can generalize a preference rather than merely copy an action sequence. A demonstrated route across one terrain reveals something about which local properties are desirable. On a new terrain, the original moves may be impossible, but the learned feature weights can still rank new candidate routes.

Hierarchical control reduces the scale of each decision. A foot-placement planner compares a manageable set of future contact locations, while a lower-level controller handles motor commands at finer time scales. Failure at either level affects the other: a beautiful high-level plan is unhelpful if its low-level actions cannot reliably be executed.

24.4 Autonomous driving and manipulation

An autonomous vehicle combines positioning, perception, planning, and feedback steering. Laser height differences can flag obstacles, but raw measurements contain errors. A temporal probabilistic model combines uncertain position and repeated sensor observations, reducing spurious detections relative to isolated threshold tests. Camera-based perception can learn from other sensors' reliable labels, an example of self-supervision. Neural semantic segmentation assigns a class to each image pixel.

Robotic laundry, object manipulation, and learning several physical skills add high-dimensional perception, contact dynamics, and generalization to the control problem. Learned visual and control representations connect observations to action, while interaction supplies data unavailable from static labeled images alone. These examples show why sensing uncertainty, reward design, computation, and learning must be addressed together.

In obstacle detection, a height difference can result from an object or from measurement and pose error. Temporal inference considers whether a sequence of observations is more consistent with a persistent obstacle than with noise. Its benefit comes from modeling how errors and states relate across observations; blindly averaging contradictory readings is not the same operation.

Self-supervision uses a reliable signal from one part of the system to train another. A range sensor can identify nearby drivable ground and provide labels for camera appearance, allowing vision to extend beyond the range sensor's immediate coverage. The resulting predictor still depends on the quality and coverage of its training labels.

The central engineering difficulty is keeping the components' assumptions compatible. A planner needs a transition model that reflects what the controller can do, and a controller needs an estimate containing the variables that affect its dynamics. Better optimization cannot fix a state representation that leaves out essential information.

Source. Dan Klein, Pieter Abbeel, and Anca Dragan, Introduction to Artificial Intelligence, Fall 2018, Lecture 24.

Lecture 25: Advanced Topics and Final Contest

25.1 Cooperative game strategies

Cooperative Pac-Man requires an agent to collect food while coordinating with another agent and responding to ghosts. The contest strategies illustrate several ways to combine the course's tools.

A feature-based reflex policy can weigh distance to food, teammate separation, and ghost danger. Seeking food farther from a nearby teammate reduces duplicated effort. Approximate Q-learning can instead learn weights for distance and score features, including a term that discourages deadlock.

A reward-density strategy first scores areas by food concentration, lowers scores where a teammate is likely to go, discounts distant targets, and penalizes nearby ghosts. Particle filtering maintains beliefs about teammate positions. A* selects routes to attractive areas when safe, while minimax handles immediate ghost danger. The central lesson is that estimation, search, local evaluation, and learning can serve different parts of one policy.

The components solve different failures of a purely local food-seeking rule. Teammate separation avoids both agents chasing the same reward. Ghost features discourage a locally attractive move that is unsafe. An exploration term can discourage repeatedly revisiting the same state without progress. A belief about an unobserved teammate matters because coordination depends on what that teammate is likely to do, not only on its last visible position.

A reward-density map expresses a spatial objective, while A* supplies a route toward a selected target. Switching to adversarial search near a ghost changes how immediate uncertainty is modeled. These choices must share a consistent notion of what is valuable, or one component can undo another's progress. A complicated collection of methods is useful only when each addresses a concrete part of the task.

25.2 Speech as sequential inference

Speech recognition maps an acoustic waveform to words. Short-time spectral analysis describes energy across frequencies. Vocal-cord vibration supplies harmonics; the vocal tract amplifies selected frequency bands according to its shape. Different speakers, sounds, speaking rates, and transitions between sounds make matching waveforms directly unreliable.

An HMM uses acoustic feature vectors as observations and a pronunciation cursor as its hidden state: a position within a word's sequence of sounds. Emissions model how likely an acoustic vector is for that sound. Transitions advance within words and between words, allowing variation in duration and pronunciation.

A language model scores transitions between words. A bigram uses $P(w_t \mid w_{t-1})$; a trigram uses $P(w_t \mid w_{t-2}, w_{t-1})$. More context captures additional correlations but requires more data or parameter sharing. For example, the probability of "door" after "close the" can be far larger than after "the" alone.

Decoding finds the most likely state sequence given the acoustics, then reads the words from that sequence. This is a Viterbi query rather than a sequence of independent classifications:

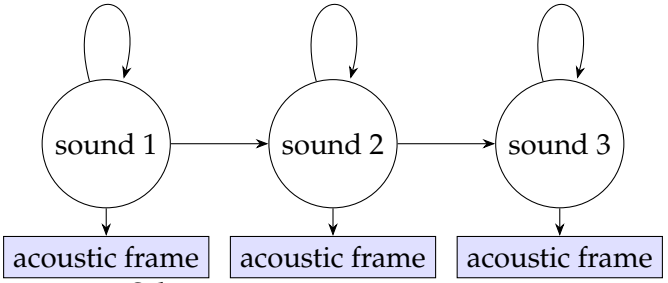
$$\hat{x}_{1:T} = \arg\max_{x_{1:T}} P(x_1)P(e_1 \mid x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1})P(e_t \mid x_t).$$

Acoustic and language models contribute different evidence. Neural networks can parameterize these models and learn richer representations than fixed local tables. Robust recognition at scale still differs from full contextual understanding of language.

The hidden state must carry a position within a pronunciation, rather than just a word label for the entire utterance. A word lasts many acoustic frames, and different frames correspond to different sounds or transitions. Self-transitions allow a state to persist for multiple frames; forward transitions advance through the pronunciation.

The acoustic model scores how well a sound state explains the current frame. The language model scores how plausible a word continuation is. A locally plausible sound match can belong to an implausible word sequence, while a common phrase can have poor acoustic support. Decoding balances both kinds of evidence.

Increasing language-model order preserves more context but fragments counts. The contexts for a trigram are word pairs, so many specific pairs may be rare even in a large dataset. Smoothing or learned parameter sharing is therefore needed to avoid treating every unseen sequence as impossible. This is the same sparse-count problem seen in naive Bayes, now applied to a sequence model.



Schematic pronunciation states;
a self-loop allows variable duration.

Figure 27: Speech decoding follows a path through sound states that explain the frames.

Viterbi stores the best prefix ending at each pronunciation state. Many different word sequences

can share possible sound prefixes, so searching a trellis reuses computation rather than expanding every full sequence independently. Taking logarithms turns path-probability products into sums, which makes very long sequences easier to score numerically.

A neural acoustic or language model changes how probabilities are estimated. It does not remove the distinction between observed acoustics, hidden linguistic structure, and a decision about the final transcription. Those roles are useful for understanding what each part of the system contributes.

25.3 Connecting inference to action

Physical Pac-Man demonstrations and a robot crawler connect simulation to sensing and actuation. A policy that works in a model must receive useful observations and issue executable actions in the physical system. The same division of work recurs throughout AI: represent uncertainty, estimate the state, compare consequences under an objective, and learn missing models or representations from experience.

Moving from a simulator to a physical agent exposes model errors. A planned motion may slip, a sensor can miss an object, and an action can take longer than expected. Repeated observation closes the loop: the agent updates what it believes and replans or applies a policy to the situation actually reached. The inference and decision methods remain useful because they make these uncertainties and consequences explicit.

Source. Dan Klein and Pieter Abbeel, Introduction to Artificial Intelligence, Fall 2018, Lecture 25.